

Manuale tecnico

Indice:

1. .vscode
 - a. Launch.json
 - b. Setting.json
2. Database
 - a. Database
 - b. Michelin_my_maps
3. Lib
 - a. Javafx-sdk-21.0.2
 - b. Mysql-connector-j-9.7.0
 - c. Gson-2.10.1
4. Src
 - a. Client
 - i. Controller
 1. GestoreRistorantiController
 2. HomeController
 3. LoginController
 4. RecensioniRistoranteController
 5. RecensioniUtenteController
 6. RegisterController
 7. RicercaRistorantiController
 8. RispondiRecensioneController
 9. RisposteGestoreController
 10. RistoranteDettagliController
 11. ScriviRecensioneController

- ii. Gui
 - 1. GestoreRistorantiView
 - 2. HomeView
 - 3. LoginView
 - 4. ModificaRecensioneDialog
 - 5. PreferitiView
 - 6. PreferitoRow
 - 7. RecensioneRistoranteRow
 - 8. RecensioneRow
 - 9. RecensioniRistoranteView
 - 10. RecensioniUtenteView
 - 11. RegisterView
 - 12. RicercaRistorantiView
 - 13. RispondiRecensioneView
 - 14. RispostaRow
 - 15. Utente
- iii. Model
 - 1. RecensioneDTO
 - 2. RistoranteDTO
 - 3. UtenteDTO
- iv. Net
 - 1. ClientConnection
 - 2. Request
 - 3. Response
- v. Common
 - 1. MessageType
 - 2. Request
 - 3. Response
- b. ClientMain
- c. Server
 - i. Connection
 - 1. ClientHandler
 - 2. ServerListener
 - 3. ServerThreadPool
 - ii. Dao
 - 1. PreferitiDAO
 - 2. RecensioneDAO

3. RistoranteDAO
4. RistoranteUtenteDAO
5. UtenteDAO

iii. Model

1. Recensione
2. Risposta
3. Ristorante
4. Utente

iv. Service

1. PreferitiService
2. RecensioneService
3. RistoranteService
4. UtenteService

v. Utils

1. DatabasePopulator
2. DBConnectionPool

d. ServerMain

.vscode

Launch.json

```
1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "type": "java",
6       "name": "DatabasePopulator",
7       "request": "launch",
8       "mainClass": "server.utils.DatabasePopulator",
9       "projectName": "lab_8_35d6ae79"
10    },
11    {
12      "type": "java",
13      "name": "ServerMain",
14      "request": "launch",
15      "mainClass": "server.ServerMain",
16      "projectName": "lab_8_35d6ae79"
17    },
18    {
19      "type": "java",
20      "name": "Run Client",
21      "request": "launch",
22      "mainClass": "client.ClientMain",
23      "vmArgs": "--module-path \\\"C:/Users/Fil/Desktop/UNI/lab_8/lib/javafx-sdk-21.0.2/lib\\\" --add-modules javafx.controls,javafx.fxml"
24    }
25  ]
26 }
```

Introduzione generale

Il file `launch.json` è un componente di configurazione di **Visual Studio Code** che definisce come avviare e fare il debug di applicazioni Java.

Serve per dire all'ambiente di sviluppo *quale classe eseguire, con quali parametri, in quale progetto e con quali impostazioni della JVM*.

In pratica, è il “manuale di avvio” del tuo workspace Java.

Struttura del file

Il file è scritto in formato **JSON** e contiene due sezioni principali:

- `"version": "0.2.0"` → indica la versione del formato di configurazione usata da VS Code.

`"configurations": [...]` → è un array che elenca tutte le configurazioni di esecuzione disponibili.

- Ogni configurazione rappresenta un diverso punto d'ingresso del progetto (ad esempio server, client, o script di popolamento del database).

Campi principali di ogni configurazione

Ogni blocco dentro `"configurations"` ha parametri specifici:

- `"type": "java"` → specifica che il debugger deve usare l'estensione Java di VS Code.
- `"name"` → è il nome che appare nel menu "Run and Debug".
- `"request": "launch"` → indica che il programma deve essere avviato (non collegato a un processo già in esecuzione).
- `"mainClass"` → è il percorso completo della classe Java contenente il metodo `main()` da eseguire.
- `"projectName"` → identifica il progetto o modulo a cui appartiene la classe.
- `"vmArgs"` → (opzionale) consente di passare argomenti alla JVM, come moduli o percorsi di librerie.

Analisi delle tre configurazioni presenti

DatabasePopulator

Avvia la classe `server.utils.DatabasePopulator`.

Serve per popolare il database con i dati del file CSV Michelin.

Non richiede argomenti particolari, ma deve essere eseguita dopo aver configurato correttamente la connessione MySQL.

ServerMain

Avvia la classe `server.ServerMain`.

È il punto d'ingresso del server dell'applicazione: gestisce le connessioni, le richieste dei client e l'interfaccia con il database.

Questa configurazione è utile per avviare il backend in modalità debug.

Run Client

Avvia la classe `client.ClientMain`.

Include nel campo "`vmArgs`" i parametri necessari per caricare **JavaFX**, indispensabile per la GUI del client:

bash

```
--module-path "C:/Users/Fil/Desktop/UNI/lab_B/lib/javafx-sdk-21.0.2/lib" --add-modules  
javafx.controls,javafx.fxml
```

Senza questi argomenti, la parte grafica non si avvierebbe correttamente.

Funzionamento pratico

Quando premi **F5** o scegli una configurazione dal menu "Run and Debug":

1. VS Code legge il file `launch.json`.
2. Carica la configurazione selezionata.
3. Avvia la JVM con la classe indicata.
4. Se hai impostato breakpoint, il debugger si attiva automaticamente e puoi analizzare il flusso del programma.

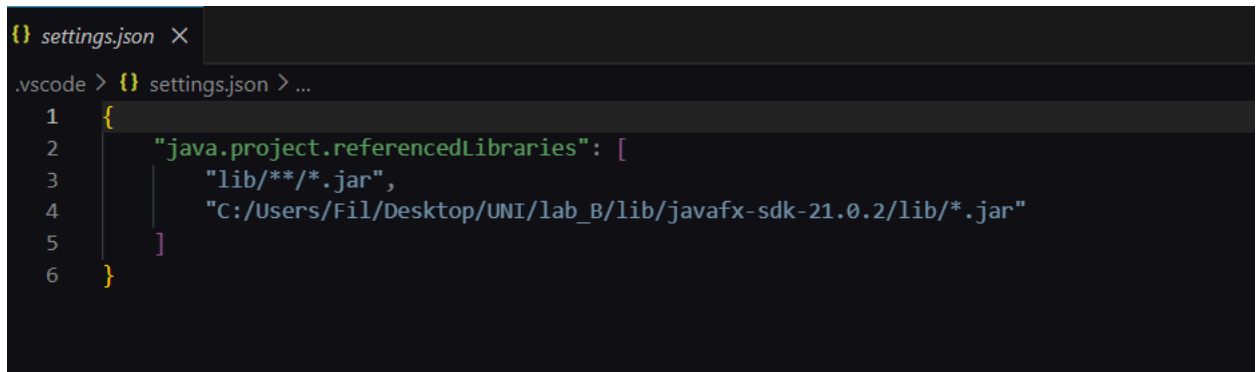
Conclusione

In sintesi, il file `launch.json` è la **configurazione di avvio automatica** del tuo ambiente Java:

definisce come eseguire il populator, il server e il client, con le giuste impostazioni per la JVM e per JavaFX.

Grazie a queste configurazioni, puoi avviare ogni componente del progetto con un solo clic, senza dover digitare comandi manuali nel terminale.

Setting.json



```
{
  "java.project.referencedLibraries": [
    "lib/**/*.jar",
    "C:/Users/Fil/Desktop/UNI/lab_B/lib/javafx-sdk-21.0.2/lib/*.jar"
  ]
}
```

Funzione del file

Il file `settings.json` serve a **configurare le impostazioni del progetto Java** all'interno di Visual Studio Code.

In particolare, la chiave `"java.project.referencedLibraries"` indica **quali librerie esterne** devono essere incluse nel classpath del progetto, cioè rese disponibili al compilatore e al runtime.

Analisi riga per riga

`"java.project.referencedLibraries"`

È la proprietà che dice all'estensione Java di VS Code dove cercare i file `.jar` da includere nel progetto.

`"lib/**/*.jar"`

Questa riga include **tutti i file JAR** presenti nella cartella `lib` e nelle sue sottocartelle.

Il doppio asterisco `**` significa "ricorsivo": quindi se hai `lib/mysql/mysql-connector.jar`, verrà incluso automaticamente.

`"C:/Users/Fil/Desktop/UNI/lab_B/lib/javafx-sdk-21.0.2/lib/*.jar"`

Questa riga aggiunge **tutte le librerie JavaFX** necessarie per far funzionare la GUI del client.

Il percorso punta alla cartella `lib` del tuo SDK JavaFX, dove si trovano i moduli come `javafx.controls.jar` e `javafx.fxml.jar`.

In pratica

Quando compili o esegui il progetto:

- VS Code legge questo file.
- Aggiunge tutte le librerie elencate al classpath.
- Permette al compilatore di risolvere correttamente le classi esterne (es. `javafx.scene.*`, `java.sql.*`, ecc.).
- Evita errori come *“package javafx.controls does not exist”*.

Conclusione

In sintesi, questo `settings.json` definisce **le dipendenze esterne del tuo progetto Java**.

Grazie a queste righe, VS Code sa dove trovare le librerie necessarie — sia quelle generiche nella cartella `lib`, sia quelle specifiche di **JavaFX** — garantendo che il progetto si compili e si esegua correttamente senza errori di classpath.

Database

Database

Funzione del file

Il file SQL definisce **l'intera struttura del database** utilizzato dall'applicazione.

Il suo scopo è:

- creare tutte le tabelle necessarie
- impostare i vincoli di integrità
- definire le relazioni tra utenti, ristoranti, recensioni, risposte e preferiti
- garantire che il database sia coerente e pronto all'uso

Il file include anche istruzioni preliminari (modalità SQL, transazione, fuso orario) e la rimozione ordinata delle tabelle esistenti.

Analisi sezione per sezione

Impostazioni iniziali

Queste righe configurano l'ambiente SQL prima della creazione delle tabelle.

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";

- Impedisce che il valore 0 venga assegnato automaticamente ai campi AUTO_INCREMENT.

START TRANSACTION;

- Avvia una transazione per garantire che tutte le operazioni vengano applicate in modo atomico.

SET time_zone = "+00:00";

- Imposta il fuso orario del database su UTC.

Eliminazione delle tabelle esistenti

Le istruzioni **DROP TABLE IF EXISTS** eliminano eventuali tabelle già presenti.

L'ordine è importante:

si eliminano prima le tabelle **dipendenti** (che hanno foreign key), poi quelle **principali**.

Questo evita errori di vincolo.

Tabella *utenti*

Contiene i dati degli utenti registrati.

Campi principali:

- **id**: chiave primaria, AUTO_INCREMENT
- **nome, email, password**: dati anagrafici e di accesso
- **ruolo**: definisce il tipo di utente (es. admin, gestore, cliente)

Vincoli:

- **PRIMARY KEY (id)**
- **UNIQUE (email)** per evitare duplicati

Funzione:

Gestisce autenticazione, autorizzazione e identificazione degli utenti.

Tabella *RistorantiTheKnife*

Rappresenta i ristoranti presenti nel sistema.

Campi principali:

- dati anagrafici (nome, nazione, città, indirizzo)
- coordinate geografiche (latitudine, longitudine)
- **fascia_prezzo**: livello di costo
- **delivery, prenotazione**: servizi disponibili
- **tipo_cucina**: categoria gastronomica

Funzione

:Fornisce tutte le informazioni necessarie per la visualizzazione e la ricerca dei ristoranti.

Tabella *recensioni*

Collega utenti e ristoranti tramite recensioni.

Campi:

- **id_utente** → FK verso *utenti*
- **id_ristorante** → FK verso *RistorantiTheKnife*
- **voto** con vincolo CHECK (1–5)
- **testo** e **fonte** della recensione

Vincoli:

ON DELETE CASCADE

- Se un utente o un ristorante viene eliminato, le recensioni collegate vengono rimosse automaticamente.

Funzione:

Gestisce il sistema di valutazione e feedback.

Tabella *risposte*

Contiene le risposte dei gestori alle recensioni.

Campi:

- **id_recensione** → FK verso *recensioni*
- **testo** della risposta

Vincoli:

ON DELETE CASCADE

- Se una recensione viene eliminata, anche le risposte associate vengono rimosse.

Funzione: Permette ai gestori di interagire con gli utenti.

Tabella *preferiti*

Gestisce la relazione multi-a-molti tra utenti e ristoranti.

Campi:

- **id_utente**
- **id_ristorante**
- **fonte**: indica da dove proviene il salvataggio (app, import, ecc.)

Vincoli:

- **PRIMARY KEY (id_utente, id_ristorante)**
- **ON DELETE CASCADE** su entrambe le FK

Funzione:

Permette agli utenti di salvare e recuperare rapidamente i ristoranti preferiti.

Commit finale

La transazione viene confermata con:

COMMIT;

Garantisce che tutte le tabelle vengano create correttamente e in modo definitivo.

In pratica

Quando il file viene eseguito:

1. Il database viene pulito da eventuali versioni precedenti.
2. Vengono create tutte le tabelle con i relativi vincoli.
3. Le relazioni tra utenti, ristoranti e contenuti vengono definite in modo rigoroso.
4. Il sistema è pronto per gestire autenticazione, recensioni, risposte e preferiti.

Conclusione

Questo file SQL costituisce **la base strutturale dell'intero sistema**.

Definisce:

- l'architettura dei dati
- le relazioni tra le entità
- i vincoli di integrità
- il comportamento in caso di eliminazioni

È un componente fondamentale del progetto, necessario per garantire coerenza, affidabilità e corretto funzionamento dell'applicazione.

Michelin_my_maps

Funzione del file

Il file **Michelin_my_maps.csv** contiene l'elenco completo dei ristoranti presenti nel progetto, insieme ai relativi dettagli descrittivi.

Il suo scopo è fornire una **fonte dati strutturata** da cui l'applicazione può importare informazioni aggiornate sui ristoranti, come nome, posizione geografica, categoria e altri attributi utili alla classificazione.

Contenuto del file

Il file è in formato **CSV (Comma-Separated Values)** e ogni riga rappresenta un ristorante.

Le colonne includono tipicamente:

- Nome del ristorante
- Indirizzo completo
- Città e nazione
- Coordinate geografiche (latitudine/longitudine)
- Categoria o tipologia (es. cucina, fascia prezzo)
- Eventuali note o descrizioni aggiuntive

Utilizzo operativo

In fase di importazione:

1. L'applicazione legge il file CSV.
2. Ogni riga viene interpretata come un record ristorante.
3. I dati vengono inseriti o aggiornati nel database, popolando la tabella *RistorantiTheKnife*.
4. Le coordinate permettono la visualizzazione su mappa e la ricerca geografica.
5. Le informazioni descrittive supportano funzioni come filtri, schede dettagliate e suggerimenti.

Conclusione

Il file `Michelin_my_maps.csv` rappresenta una fonte dati primaria per la configurazione iniziale o l'aggiornamento massivo dei ristoranti nel sistema. Grazie alla sua struttura

tabellare, consente un'importazione semplice, automatizzabile e coerente con il modello dati del progetto.

Lib

Javafx-sdk-21.0.2

Funzione del componente

JavaFX-sdk-21.0.2 è il pacchetto ufficiale che contiene tutte le librerie necessarie per sviluppare interfacce grafiche (GUI) in Java.

Include moduli come *javafx.controls*, *javafx.fxml*, *javafx.graphics* e altri componenti fondamentali per costruire finestre, pulsanti, layout, scene e interazioni grafiche.

Perché è stato utilizzato

Il progetto richiede una **interfaccia grafica desktop** per il client, e JavaFX rappresenta la soluzione standard e moderna per applicazioni Java non web.

In particolare, l'SDK è stato utilizzato per:

- **gestire la GUI del client** (finestre, pulsanti, form, viste)
- **caricare file FXML** per separare grafica e logica
- **supportare layout reattivi e componenti avanzati**
- **integrare CSS per la personalizzazione grafica**
- **garantire compatibilità con Java 21**, poiché la versione 21.0.2 è allineata alla piattaforma

Integrazione nel progetto

L'SDK è stato aggiunto tramite:

- configurazione del classpath in *settings.json*
- parametri `--module-path` e `--add-modules` nelle configurazioni di esecuzione
- inclusione dei file `.jar` presenti nella cartella `lib` dell'SDK

Questo permette al compilatore e al runtime di riconoscere correttamente tutti i moduli JavaFX necessari.

Conclusione

JavaFX-sdk-21.0.2 è stato utilizzato perché fornisce l'infrastruttura grafica indispensabile per il client del progetto.

Senza questo SDK, l'applicazione non potrebbe visualizzare finestre, controlli grafici o interfacce utente, generando errori di classpath e impedendo l'esecuzione della GUI.

Mysql-connector-j-9.7.0

Funzione del componente

mysql-connector-j-9.7.0 è il driver ufficiale JDBC fornito da MySQL.

La sua funzione è permettere a un'applicazione Java di **collegarsi a un database MySQL**, eseguire query, leggere dati e inviare comandi SQL.

Senza questo driver, Java non sarebbe in grado di comunicare con MySQL, poiché il linguaggio non include nativamente un connettore per questo database.

Perché è stato utilizzato

Il progetto richiede l'accesso al database per:

- registrare utenti
- salvare recensioni
- recuperare ristoranti
- gestire preferiti e risposte
- eseguire query SQL lato server e lato client

Per questo motivo è necessario includere il driver JDBC, che fornisce:

- gestione della connessione (`DriverManager.getConnection`)
- supporto ai protocolli MySQL
- compatibilità con MySQL 8 e versioni successive
- gestione sicura di query, prepared statement e transazioni

La versione **9.7.0** è una release recente, compatibile con Java 21 e ottimizzata per prestazioni e sicurezza.

Integrazione nel progetto

Il file `.jar` del connettore è stato:

- inserito nella cartella `lib` del progetto
- aggiunto al classpath tramite la proprietà `java.project.referencedLibraries`
- caricato automaticamente da VS Code durante la compilazione

Questo permette al codice Java di utilizzare classi come:

- `java.sql.Connection`
- `java.sql.PreparedStatement`
- `com.mysql.cj.jdbc.Driver`

e di stabilire connessioni verso il database MySQL del progetto.

Conclusione

mysql-connector-j-9.7.0 è un componente essenziale per il funzionamento del progetto:

senza questo driver, l'applicazione non potrebbe accedere al database, leggere o scrivere dati, né eseguire alcuna operazione SQL.

Gson-2.10.1

Funzione del componente

Gson-2.10.1 è una libreria Java sviluppata da Google che permette di **convertire oggetti Java in JSON e JSON in oggetti Java**.

È uno strumento essenziale quando l'applicazione deve:

- scambiare dati in formato JSON
- leggere file JSON
- serializzare e deserializzare strutture complesse
- comunicare con API o servizi esterni basati su JSON

La libreria è leggera, veloce e completamente compatibile con Java 21.

Perché è stato utilizzato

Nel progetto, Gson è stato utilizzato per:

- **importare dati da file JSON** (es. configurazioni, liste, strutture dati)
- **convertire oggetti Java in JSON** per salvataggi o esportazioni
- **gestire scambi di dati strutturati** tra componenti dell'applicazione
- **semplificare la manipolazione di dati complessi** senza dover scrivere parser manuali

La versione **2.10.1** è stabile, aggiornata e ottimizzata per prestazioni e sicurezza.

Integrazione nel progetto

Il file **.jar** della libreria è stato:

- inserito nella cartella **lib** del progetto
- aggiunto al classpath tramite **java.project.referencedLibraries**
- caricato automaticamente da VS Code durante la compilazione

Questo permette l'utilizzo diretto delle classi principali, come:

- `com.google.gson.Gson`
- `com.google.gson.GsonBuilder`
- `com.google.gson.JsonObject`
- `com.google.gson.reflect.TypeToken`

E consente operazioni come:

```
java
```

```
Gson gson = new Gson();
```

```
MyClass obj = gson.fromJson(jsonString, MyClass.class);
```

```
String json = gson.toJson(obj);
```

Conclusione

Gson-2.10.1 è stato utilizzato perché fornisce un sistema semplice, affidabile e veloce per gestire dati in formato JSON.

È una dipendenza fondamentale per tutte le parti del progetto che richiedono serializzazione, deserializzazione o scambio di dati strutturati.

Src —> client —> controller
GestoreRistorantiController

Funzione della classe

La classe **GestoreRistorantiController** gestisce tutta la logica di controllo della schermata dedicata ai ristoranti di un gestore.

Coordina:

- interazione tra GUI (JavaFX) e rete (ClientConnection)
- caricamento dei ristoranti associati al gestore
- apertura dei form di aggiunta e modifica
- eliminazione dei ristoranti
- apertura dei dettagli tramite doppio click

È quindi il **controller principale** per la gestione CRUD dei ristoranti lato client.

Analisi delle responsabilità principali

Gestione della vista

La classe riceve in input:

- la view **GestoreRistorantiView**
- la connessione al server **ClientConnection**
- una callback per tornare indietro
- una callback per aprire i dettagli di un ristorante

Il controller collega questi elementi e registra tutti gli handler dei pulsanti.

Inizializzazione degli handler

Il metodo `initHandlers()` associa gli eventi ai pulsanti della GUI:

- **Aggiungi** → apre il form di inserimento
- **Modifica** → apre il form precompilato
- **Elimina** → invia richiesta di cancellazione al server
- **Indietro** → esegue la callback `onGoBack`
- **Doppio click sulla tabella** → apre i dettagli del ristorante selezionato

Questo metodo centralizza tutta la logica di interazione dell'interfaccia.

Caricamento dei ristoranti del gestore

Il metodo `loadRiepilogo()`:

1. crea un JSON con l'ID del gestore loggato
2. invia al server la richiesta `VISUALIZZA_RIEPILOGO_GESTORE`
3. riceve un array JSON di ristoranti
4. aggiorna la tabella JavaFX tramite `Platform.runLater()`

Ogni ristorante viene convertito in un oggetto `RistoranteRow` e inserito nella tabella.

Funzione: **sincronizzare la GUI con i dati del server.**

Apertura del form di aggiunta

Il metodo `apriFormAggiunta()`:

- crea una nuova view `RistoranteFormView`
- apre una nuova finestra JavaFX
- istanzia un `RistoranteFormController` in modalità *aggiunta*
- aggiorna la tabella al termine dell'operazione

Serve per creare nuovi ristoranti associati al gestore.

Apertura del form di modifica

Il metodo `apriFormModifica()`:

1. verifica che un ristorante sia selezionato
2. invia al server la richiesta `VISUALIZZA_UTENTE` per ottenere i dettagli completi
3. precompila il form con i dati ricevuti
4. apre il form in modalità *modifica*, passando l'ID del ristorante

Funzione: **permettere la modifica completa dei dati del ristorante.**

Eliminazione del ristorante

Il metodo `eliminaRistorante()`:

- controlla che un ristorante sia selezionato
- invia al server la richiesta `ELIMINA_RISTORANTE`
- aggiorna la tabella se l'operazione va a buon fine

Funzione: **gestire la cancellazione remota tramite server.**

In pratica

La classe funge da ponte tra:

- **GUI** (JavaFX)
- **Server** (tramite Request/Response)
- **Modelli dati** (RistoranteRow, JSON)

Gestisce tutte le operazioni CRUD dei ristoranti del gestore e aggiorna dinamicamente la vista.

Conclusione

La classe **GestoreRistorantiController** è il cuore della gestione ristoranti lato client.

Coordina eventi, richieste al server e aggiornamenti della GUI, garantendo un flusso coerente tra interfaccia, dati e logica applicativa.

Funzione della classe

La classe **HomeController** gestisce tutta la logica della schermata principale dell'applicazione (Home).

Coordina:

- la navigazione verso le varie sezioni del client
- la gestione dello stato dell'utente loggato
- l'invio di richieste semplici al server
- la visibilità dinamica dei pulsanti in base al ruolo (utente/gestore)

È quindi il **controller centrale** che governa l'accesso alle funzionalità principali del sistema.

Analisi delle responsabilità principali

Gestione della vista Home

Il controller riceve:

- la view **HomeView**
- la connessione al server **ClientConnection**
- una serie di callback (**Runnable**) per cambiare schermata

Questo approccio mantiene la classe indipendente dalla logica di routing, che viene delegata ai callback.

Inizializzazione degli handler

Il metodo `initHandlers()` associa gli eventi ai pulsanti della Home.

Pulsanti principali

- **Cerca ristoranti** → `onGoSearch.run()`
- **Preferiti** → invia richiesta al server e apre la schermata preferiti
- **Recensioni utente** → apre la sezione recensioni personali
- **Gestione ristoranti** (solo gestore) → apre la dashboard di gestione
- **Recensioni ricevute** (solo gestore) → apre la sezione dedicata alle recensioni dei clienti
- **Login** → apre la schermata di autenticazione
- **Logout** → resetta l'utente loggato e aggiorna la visibilità dei pulsanti

Ogni pulsante è collegato a un'azione ben definita, mantenendo la Home semplice e reattiva.

Gestione della visibilità dinamica

Alla fine del costruttore viene chiamato:

```
java  
view.refreshVisibility();
```

La view decide quali pulsanti mostrare in base a:

- utente loggato o meno
- ruolo dell'utente (cliente o gestore)

Il controller quindi **non gestisce direttamente la UI**, ma la aggiorna quando necessario.

Invio di richieste semplici al server

Il metodo `sendSimple(MessageType type)` permette di inviare richieste senza parametri complessi.

Funzionamento:

1. crea un JSON vuoto
2. aggiunge automaticamente l'ID dell'utente loggato
3. invia la richiesta al server
4. stampa il messaggio di risposta

È utilizzato, ad esempio, per:

- caricare i preferiti
- aggiornare dati utente
- operazioni rapide che non richiedono payload complessi

Questo metodo centralizza la logica di comunicazione semplificata.

Logout dell'utente

Alla pressione del pulsante Logout:

- l'utente loggato viene azzerato tramite `UtenteDTO.creaUtenteLoggato(0, null, null, null)`
- la visibilità della Home viene aggiornata
- viene eseguita la callback `onLogout`

Questo garantisce un reset pulito dello stato dell'applicazione.

Metodo **refresh()**

Viene chiamato quando si ritorna alla Home da altre schermate.

Funzione:

- aggiornare la visibilità dei pulsanti
- riflettere eventuali cambiamenti di stato dell'utente

È un metodo semplice ma fondamentale per mantenere la Home coerente.

In pratica

La classe **HomeController** funge da:

- **router logico** tra le varie sezioni dell'applicazione
- **gestore dello stato utente** (login/logout)
- **intermediario verso il server** per richieste rapide
- **coordinatore della visibilità della UI**

È il punto di ingresso dell'esperienza utente e determina quali funzionalità sono accessibili.

Conclusione

La classe **HomeController** è il controller principale della schermata Home. Gestisce navigazione, stato dell'utente, richieste al server e aggiornamento della UI.

È una componente essenziale per garantire un flusso di utilizzo chiaro, ordinato e coerente con i ruoli dell'applicazione.

LoginController

Funzione della classe

La classe **LoginController** gestisce tutta la logica della schermata di accesso dell'applicazione.

Coordina:

- la validazione delle credenziali inserite dall'utente
- l'invio della richiesta di login al server
- la gestione della risposta (successo o errore)
- la navigazione verso registrazione o ritorno alla Home

È quindi il **controller responsabile dell'autenticazione** lato client.

Analisi delle responsabilità principali

Gestione della vista Login

Il controller riceve:

- la view `LoginView`
- la connessione al server `ClientConnection`
- tre callback:
 - `onLoginSuccess` → da eseguire dopo login riuscito
 - `onGoRegister` → per aprire la schermata di registrazione
 - `onGoBack` → per tornare alla schermata precedente

Questo permette di mantenere il controller indipendente dalla logica di routing.

inizializzazione degli handler

Il metodo `initHandlers()` associa gli eventi ai pulsanti della GUI:

- **Accedi** → esegue il metodo `doLogin()`
- **Registrati** → esegue la callback `onGoRegister`
- **Indietro** → esegue la callback `onGoBack`

La classe quindi collega la view alle azioni logiche corrispondenti.

Metodo `doLogin()`

È il cuore della classe: gestisce l'intero processo di autenticazione.

1. Lettura e validazione dei campi

Vengono letti email e password dalla view.

Se uno dei due è vuoto → viene mostrato un messaggio di errore.

2. Creazione della richiesta

Viene costruito un JSON:

```
java
params.addProperty("email", email);
params.addProperty("password", password);
```

e inviato al server tramite:

```
java
Request req = new Request(MessageType.LOGIN, params);
```

3. Gestione della risposta

Se il server risponde con successo:

- viene estratto il payload JSON
- viene creato l'utente loggato tramite `UtenteDTO.creaUtenteLoggato(...)`
- viene eseguita la callback `onLoginSuccess`

Se il login fallisce:

- viene stampato il messaggio di errore restituito dal server

4. Gestione degli errori di rete

In caso di eccezioni:

- viene stampato lo stack trace
- viene mostrato un messaggio di errore di connessione

Gestione dello stato utente

Dopo un login riuscito, la classe salva i dati dell'utente in:

```
java
UtenteDTO.creaUtenteLoggato(...)
```

Questo permette al resto dell'applicazione di sapere:

- chi è loggato
- quale ruolo ha (utente/gestore)
- quali funzionalità può accedere

Il controller quindi **inizializza lo stato globale dell'utente**.

PreferitiController

Funzione della classe

La classe **PreferitiController** gestisce la schermata dedicata ai ristoranti preferiti dell'utente.

Coordina:

- il caricamento dei preferiti dal server
- la visualizzazione dei dati nella tabella JavaFX
- la gestione del doppio click per aprire i dettagli del ristorante
- la navigazione verso la schermata precedente

È quindi il **controller responsabile della gestione e visualizzazione dei preferiti lato client**.

Analisi delle responsabilità principali

Gestione della vista Preferiti

Il controller riceve:

- la view **PreferitiView**
- la connessione al server **ClientConnection**
- una callback **onBack** per tornare alla schermata precedente
- una callback **onOpenRistorante** per aprire i dettagli del ristorante selezionato

Questo approccio mantiene la classe modulare e indipendente dalla logica di navigazione.

Inizializzazione degli handler

Il metodo `initHandlers()` associa gli eventi ai componenti della GUI.

Pulsante Indietro

java

```
view.getBtnIndietro().setOnAction(e -> onBackPressed());
```

Permette di tornare alla schermata precedente.

Doppio click sulla tabella

java

```
if (e.getClickCount() == 2)
```

Se l'utente fa doppio click su un ristorante:

- viene recuperata la riga selezionata (`PreferitoRow`)
- viene chiamata la callback `onOpenRistorante` passando:
 - **ID del ristorante**
 - **fonte** (es. "UTENTE", "GOOGLE", ecc.)

Funzione: **aprire la scheda dettagliata del ristorante preferito.**

Caricamento dei preferiti

Il metodo `loadPreferiti()` è il cuore della classe.

1. Creazione dei parametri

Viene creato un JSON contenente l'ID dell'utente loggato:

java

```
params.addProperty("idUser", UtenteDTO.getUtenteLoggato().getId());
```

2. Invio della richiesta al server

Viene inviata la richiesta:

java

```
new Request(MessageType.VISUALIZZA_PREFERITI, params)
```

3. Gestione della risposta

Se la risposta è valida:

- viene estratto l'array JSON "preferiti"
- la tabella viene svuotata
- ogni elemento viene convertito in un oggetto `PreferitoRow`

4. Aggiornamento della GUI

L'aggiornamento avviene tramite:

java

```
Platform.runLater(...)
```

per garantire la modifica sicura della UI da thread JavaFX.

Creazione delle righe della tabella

Ogni ristorante preferito viene trasformato in:

java

```
new PreferitoRow(
```

```
    id,
```

```
    nome,
```

```
    indirizzo,
```

```
    citta,
```

```
    nazione,
```

```
    tipoCucina,
```

```
    fasciaPrezzo,
```

```
    delivery,
```

```
    prenotazione,
```

```
    fonte
```

```
)
```

La presenza del campo **fonte** permette di distinguere l'origine del preferito (es. aggiunto dall'utente, importato, sincronizzato).

In pratica

La classe **PreferitiController** funge da:

- **intermediario** tra GUI e server
- **gestore della tabella dei preferiti**
- **router** verso la schermata dei dettagli ristorante
- **coordinatore** della navigazione e dell'aggiornamento della UI

È una componente essenziale per offrire all'utente un accesso rapido ai ristoranti salvati.

Conclusione

La classe **PreferitiController** implementa l'intero flusso di gestione dei ristoranti preferiti.

Carica i dati dal server, li visualizza nella tabella, gestisce il doppio click per aprire i dettagli e permette di tornare alla schermata precedente.

È un controller fondamentale per migliorare l'esperienza utente e rendere immediata la consultazione dei ristoranti salvati.

RecensioniRistoranteController

Funzione della classe

La classe **RecensioniRistoranteController** gestisce la schermata che mostra tutte le recensioni ricevute dai ristoranti di un gestore.

Coordina:

- il caricamento delle recensioni dal server
- la visualizzazione dei dati nella tabella JavaFX
- la gestione del pulsante di chiusura della finestra

È quindi il **controller responsabile della consultazione delle recensioni ricevute** lato gestore.

Analisi delle responsabilità principali

Gestione della vista RecensioniRistorante

Il controller riceve:

- la view **RecensioniRistoranteView**
- la connessione al server **ClientConnection**
- una callback **onClose** per chiudere la finestra

Questo mantiene la classe indipendente dalla logica di navigazione e permette un riutilizzo più semplice.

Inizializzazione degli handler

Il metodo `initHandlers()` collega il pulsante “Chiudi” alla callback:

```
java  
view.getBtnChiudi().setOnAction(e -> onClose.run());
```

Funzione: **chiudere la finestra e tornare alla schermata precedente.**

Caricamento delle recensioni

Il metodo `loadRecensioni()` è il cuore della classe.

1. Creazione dei parametri

Viene creato un JSON contenente l'ID del gestore loggato:

```
java  
params.addProperty("idGestore", UtenteDTO.getUtenteLoggato().getId());
```

Questo permette al server di filtrare solo le recensioni dei ristoranti gestiti dall'utente.

2. Invio della richiesta al server

La richiesta viene inviata tramite:

```
java  
new Request(MessageType.VISUALIZZA_RECENSIONI_GESTORE, params)
```

3. Gestione della risposta

Se la risposta è valida:

- viene estratto l'array "recensioni"
- la tabella viene svuotata
- ogni recensione viene convertita in un oggetto `RecensioneRistoranteRow`

4. Aggiornamento della GUI

L'aggiornamento avviene tramite:

```
java  
Platform.runLater(...)
```

per garantire la modifica sicura della UI.

Creazione delle righe della tabella

Ogni recensione viene trasformata in:

```
java  
new RecensioneRistoranteRow(  
    idRecensione,  
    idRistorante,  
    idUtente,  
    voto,  
    testo,  
    data,  
    fonte  
)
```

I campi principali includono:

- **id recensione** → utile per eventuali risposte o gestione avanzata
- **id ristorante** → identifica a quale ristorante appartiene
- **id utente** → autore della recensione
- **voto e testo** → contenuto della recensione
- **data** → quando è stata lasciata
- **fonte** → indica da dove proviene (es. UTENTE, GOOGLE, ecc.)

Questo permette una visualizzazione completa e informativa.

In pratica

La classe **RecensioniRistoranteController** funge da:

- **intermediario** tra GUI e server
- **gestore della tabella delle recensioni ricevute**
- **coordinatore** dell'aggiornamento della UI
- **controller dedicato alla consultazione dei feedback dei clienti**

È una componente essenziale per permettere al gestore di monitorare la reputazione dei propri ristoranti.

Conclusione

La classe **RecensioniRistoranteController** implementa l'intero flusso di caricamento e visualizzazione delle recensioni ricevute dai ristoranti gestiti.

Gestisce la comunicazione col server, aggiorna la tabella e permette di chiudere la schermata in modo semplice e intuitivo.

RecensioniUtenteController

Funzione della classe

La classe **RecensioniUtenteController** gestisce la schermata dedicata alle recensioni scritte dall'utente.

Coordina:

- il caricamento delle recensioni dal server
- la visualizzazione nella tabella JavaFX
- la modifica di una recensione tramite popup dedicato
- l'eliminazione di una recensione
- la navigazione verso la schermata precedente

È quindi il **controller responsabile della gestione delle recensioni personali** lato client.

Analisi delle responsabilità principali:

Gestione della vista RecensioniUtente

Il controller riceve:

- la view **RecensioniUtenteView**
- la connessione al server **ClientConnection**
- una callback **onGoBack** per tornare alla schermata precedente

Questo mantiene la classe modulare e separata dalla logica di routing.

Inizializzazione degli handler

Il metodo **initHandlers()** collega i pulsanti della GUI alle rispettive azioni:

- **Indietro** → ritorna alla schermata precedente
- **Modifica** → apre il popup per modificare la recensione selezionata
- **Elimina** → elimina la recensione selezionata

Ogni pulsante è associato a un metodo dedicato, mantenendo la logica chiara e separata.

Modifica di una recensione

Il metodo `modificaRecensione()` gestisce l'intero flusso di modifica.

. Selezione della recensione

Viene verificato che l'utente abbia selezionato una riga nella tabella.

Apertura del popup

Viene creato un `ModificaRecensioneDialog`, precompilato con:

- commento attuale
- voto attuale

L'utente può confermare o annullare.

Invio della richiesta al server

Se confermato:

- viene creato un JSON con `idRecensione`, `voto` e `testo`
- viene inviata la richiesta `MODIFICA_RECENSIONE`

Aggiornamento della tabella

Se la modifica va a buon fine:

- viene ricaricata la lista delle recensioni tramite `loadRecensioni()`

Caricamento delle recensioni dell'utente

Il metodo `loadRecensioni()`:

Prepara i parametri

Inserisce l'ID dell'utente loggato:

```
java
params.addProperty("idUser", UtenteDTO.getUtenteLoggato().getId());
```

Invia la richiesta

La richiesta è:

```
java
MessageType.VISUALIZZA_RECENSIONI_UTENTE
```

Gestisce la risposta

Se la risposta è valida:

- viene estratto l'array "`recensioni`"
- la tabella viene svuotata
- ogni elemento viene convertito in un `RecensioneRow`

Aggiorna la GUI

L'aggiornamento avviene tramite:

```
java
Platform.runLater(...)
```

per garantire la sicurezza del thread JavaFX.

Eliminazione di una recensione

Il metodo `eliminaRecensione()`:

1. verifica che una recensione sia selezionata
2. crea un JSON con `idRecensione`
3. invia la richiesta `ELIMINA_RECENSIONE`
4. se l'operazione ha successo, ricarica la tabella

Funzione: **gestire la cancellazione remota tramite server.**

In pratica

La classe **RecensioniUtenteController** funge da:

- **intermediario** tra GUI e server
- **gestore della tabella delle recensioni personali**
- **controller per modifica ed eliminazione**
- **coordinatore della navigazione**

È una componente essenziale per permettere all'utente di gestire i propri feedback.

Conclusione

La classe **RecensioniUtenteController** implementa l'intero flusso di gestione delle recensioni scritte dall'utente.

Carica i dati dal server, li visualizza nella tabella, permette la modifica tramite popup e gestisce l'eliminazione.

È un controller fondamentale per la parte di interazione personale dell'utente con il sistema.

RegisterController

Funzione della classe

La classe **RegisterController** gestisce l'intero flusso di registrazione di un nuovo utente.

Coordina:

- la lettura e validazione dei dati inseriti nel form
- la costruzione della richiesta di registrazione verso il server
- la gestione della risposta (successo o errore)
- la navigazione verso la schermata precedente o successiva

È quindi il **controller responsabile della creazione di nuovi account** lato client.

Analisi delle responsabilità principali:

Gestione della vista Register

Il controller riceve:

- la view **RegisterView**
- la connessione al server **ClientConnection**
- una callback **onGoBack** per tornare alla schermata precedente
- una callback **onRegisterSuccess** da eseguire dopo una registrazione riuscita

Questo approccio mantiene la classe indipendente dalla logica di routing e più facilmente riutilizzabile.

Inizializzazione degli handler

Il metodo `initHandlers()` collega i pulsanti della GUI alle rispettive azioni:

- **Indietro** → ritorna alla schermata precedente
- **Registrati** → esegue il metodo `doRegister()`

La logica rimane chiara e separata dalla view.

Registrazione di un nuovo utente

Il metodo `doRegister()` gestisce l'intero processo di registrazione.

Lettura dei campi del form

Vengono letti:

- username
- email
- password
- conferma password
- ruolo (CLIENTE o GESTORE)

Il ruolo viene determinato tramite checkbox:

java

```
String ruolo = view.getChkGestore().isSelected() ? "GESTORE" : "CLIENTE";
```

Validazione dei dati

Il controller verifica:

- che tutti i campi siano compilati
- che password e conferma coincidano

In caso contrario, viene mostrato un messaggio di errore.

Preparazione della richiesta

Viene creato un JSON con i dati dell'utente:

```
java  
  
params.addProperty("username", username);  
  
params.addProperty("email", email);  
  
params.addProperty("password", password);  
  
params.addProperty("ruolo", ruolo);
```

La richiesta inviata è:

```
java  
  
new Request(MessageType.REGISTRAZIONE, params)
```

Gestione della risposta del server

Se la registrazione va a buon fine:

- viene stampato un messaggio di conferma
- viene eseguita la callback `onRegisterSuccess`

Se fallisce:

- viene mostrato il messaggio di errore restituito dal server
-

Gestione degli errori di rete

In caso di eccezioni:

- viene stampato lo stack trace
- viene mostrato un messaggio di errore di connessione

In pratica

La classe **RegisterController** funge da:

- **validatore** dei dati inseriti dall'utente
- **intermediario** tra GUI e server
- **gestore del flusso di registrazione**
- **coordinatore della navigazione**

È una componente essenziale per permettere la creazione di nuovi account nel sistema.

Conclusione

La classe **RegisterController** implementa l'intero processo di registrazione:

legge i dati dal form, li valida, invia la richiesta al server e gestisce la risposta.

È un controller fondamentale per l'accesso al sistema e per la gestione degli utenti.

RicercaRistorantiController

Funzione della classe

La classe **RicercaRistorantiController** gestisce l'intero flusso di ricerca dei ristoranti all'interno dell'applicazione.

Coordina:

- la lettura dei filtri inseriti dall'utente
- la costruzione della richiesta di ricerca verso il server
- la visualizzazione dei risultati nella tabella
- l'apertura dei dettagli di un ristorante tramite doppio click
- la navigazione verso la schermata precedente

È quindi il **controller responsabile della ricerca avanzata dei ristoranti** lato client.

Analisi delle responsabilità principali :

Gestione della vista RicercaRistoranti

Il controller riceve:

- la view **RicercaRistorantiView**
- la connessione al server **ClientConnection**
- una callback **onBack** per tornare alla schermata precedente
- una callback **onOpenRistorante** per aprire i dettagli del ristorante selezionato

Questo mantiene la classe modulare e indipendente dalla logica di routing.

Inizializzazione degli handler

Il metodo `initHandlers()` collega gli elementi della GUI alle rispettive azioni:

- **Indietro** → ritorna alla schermata precedente
- **Cerca** → esegue il metodo `cerca()`
- **Doppio click sulla tabella** → apre i dettagli del ristorante selezionato

Il doppio click utilizza:

```
java  
onOpenRistorante.accept(r.getId(), r.getFonte());
```

per passare ID e fonte del ristorante.

Ricerca dei ristoranti

Il metodo `cerca()` è il cuore della classe.

1. Lettura dei filtri dalla view

Vengono raccolti tutti i parametri impostati dall'utente:

- testo libero (`query`)
- tipo di cucina
- località
- fascia di prezzo minima e massima
- disponibilità delivery
- disponibilità prenotazione
- numero minimo di stelle

Questi filtri permettono una ricerca molto precisa.

2. Creazione del payload JSON

I parametri vengono inseriti in un oggetto JSON:

```
java
payload.addProperty("query", query);
payload.addProperty("tipoCucina", tipoCucina);
payload.addProperty("localita", localita);
payload.addProperty("prezzoMin", prezzoMin);
payload.addProperty("prezzoMax", prezzoMax);
payload.addProperty("delivery", delivery);
payload.addProperty("prenotazione", prenotazione);
payload.addProperty("stelleMin", stelleMin);
```

3. Invio della richiesta al server

La richiesta viene inviata tramite:

```
java
new Request(MessageType.CERCA_RISTORANTI, payload)
```

4. Gestione della risposta

Se la risposta non è valida:

```
java
if (!res.isOk()) {
    System.out.println("Errore: " + res.getMessage());
    return;
}
```

Se è valida:

- viene estratto l'array "**ristoranti**"
- la tabella viene svuotata
- ogni ristorante viene convertito in un **RistoranteRow**

5. Popolamento della tabella

Ogni risultato viene aggiunto alla tabella:

```
java
new RistoranteRow(
    id,
    nome,
    tipo_cucina,
    indirizzo,
    fonte
)
```

La presenza del campo **fonte** permette di distinguere l'origine del ristorante (es. UTENTE, GOOGLE, MICHELIN).

In pratica

La classe **RicercaRistorantiController** funge da:

- **intermediario** tra GUI e server
- **gestore dei filtri di ricerca**
- **controller della tabella dei risultati**
- **router** verso la schermata dei dettagli ristorante

È una componente essenziale per permettere all'utente di trovare rapidamente ristoranti in base ai propri criteri.

Conclusione

La classe **RicercaRistorantiController** implementa l'intero flusso di ricerca dei ristoranti:

legge i filtri, invia la richiesta al server, gestisce la risposta e aggiorna la tabella.

Permette inoltre di aprire i dettagli tramite doppio click e di tornare alla schermata precedente.

È un controller fondamentale per la funzionalità di ricerca dell'applicazione

RispondiRecensioneController

Funzione della classe

La classe **RispondiRecensioneController** gestisce la finestra dedicata all'invio di una risposta da parte del gestore a una recensione ricevuta.

Coordina:

- la visualizzazione del testo originale della recensione
- la raccolta della risposta inserita dal gestore
- l'invio della risposta al server
- la chiusura della finestra dopo l'operazione

È quindi il **controller responsabile della gestione delle risposte alle recensioni** lato gestore.

Analisi delle responsabilità principali:

Gestione della vista RispondiRecensione

Il controller riceve:

- la view **RispondiRecensioneView**
- la connessione al server **ClientConnection**
- una callback **onSuccess** da eseguire dopo l'invio o l'annullamento
- l'ID della recensione a cui rispondere
- il testo della recensione originale

Il testo della recensione viene immediatamente mostrato nella view tramite:

java

```
view.setRecensioneText(testoRecensione);
```

Inizializzazione degli handler

Il metodo `initHandlers()` collega i pulsanti della GUI:

- **Invia** → esegue `inviaRisposta()`
- **Annulla** → chiude la finestra tramite `onSuccess.run()`

Questo mantiene la logica semplice e chiara.

Invio della risposta

Il metodo `inviaRisposta()` gestisce l'intero flusso di invio.

1. Lettura del testo della risposta

Viene letto il contenuto inserito dal gestore:

```
java
String risposta = view.getTxtRisposta().getText().trim();
```

Se il campo è vuoto → viene mostrato un messaggio di errore.

2. Preparazione della richiesta

Viene creato un JSON con:

- `idRecensione` → identifica la recensione originale
- `risposta` → testo della risposta del gestore

```
java
params.addProperty("idRecensione", idRecensione);
params.addProperty("risposta", risposta);
```

3. Invio al server

La richiesta viene inviata tramite:

```
java  
new Request(MessageType.RISPONDI_RECENSIONE, params)
```

4. Gestione della risposta

Se il server conferma:

- viene stampato “Risposta inviata”
- viene eseguita la callback `onSuccess` per chiudere la finestra e aggiornare la schermata precedente

Se fallisce:

- viene mostrato il messaggio di errore restituito dal server

5. Gestione degli errori di rete

In caso di eccezioni:

- viene stampato lo stack trace
- viene mostrato un messaggio di errore generico

In pratica

La classe **RispondiRecensioneController** funge da:

- **intermediario** tra GUI e server
- **gestore del form di risposta**
- **controller dedicato alla comunicazione gestore → utente**

- **coordinatore della chiusura e aggiornamento della UI**

È una componente essenziale per permettere ai gestori di interagire con i clienti e mantenere attiva la comunicazione.

Conclusione

La classe **RispondiRecensioneController** implementa l'intero flusso di risposta alle recensioni:

mostra la recensione originale, raccoglie la risposta del gestore, invia la richiesta al server e chiude la finestra al termine.

È un controller fondamentale per la gestione della reputazione e del dialogo con gli utenti.

RisposteGestoreController

Funzione della classe

La classe **RisposteGestoreController** gestisce la schermata in cui il gestore può visualizzare tutte le recensioni ricevute e rispondere direttamente a ciascuna di esse.

Coordina:

- il caricamento delle recensioni dei ristoranti gestiti
- la visualizzazione nella tabella JavaFX
- l'apertura del form per rispondere a una recensione
- la navigazione verso la schermata precedente

È quindi il **controller responsabile della gestione delle risposte alle recensioni** lato gestore.

Analisi delle responsabilità principali:

Gestione della vista RisposteGestore

Il controller riceve:

- la view `RisposteGestoreView`
- la connessione al server `ClientConnection`
- una callback `onGoBack` per tornare alla schermata precedente

Questo mantiene la classe modulare e separata dalla logica di routing.

Inizializzazione degli handler

Il metodo `initHandlers()` collega i pulsanti della GUI:

- **Indietro** → ritorna alla schermata precedente
- **Rispondi** → apre la finestra per rispondere alla recensione selezionata

La logica rimane chiara e ben separata dalla view.

Caricamento delle recensioni ricevute

Il metodo `loadRecensioni()` è il cuore della classe.

1. Preparazione dei parametri

Viene inserito l'ID del gestore loggato:

```
java
params.addProperty("idGestore", UtenteDTO.getUtenteLoggato().getId());
```

2. Invio della richiesta

La richiesta è:

```
java
new Request(MessageType.VISUALIZZA_RECENSIONI_GESTORE, params)
```

3. Gestione della risposta

Se la risposta è valida:

- viene estratto l'array "`recensioni`"
- la tabella viene svuotata
- ogni recensione viene convertita in un `RispostaRow`

4. Aggiornamento della GUI

L'aggiornamento avviene tramite:

```
java
Platform.runLater(...)
```

per garantire la sicurezza del thread JavaFX.

Creazione delle righe della tabella

Ogni recensione viene trasformata in:

```
java
new RispostaRow(
    idRecensione,
    idUtente,
    voto,
    testo,
    risposta
)
```

I campi principali includono:

- **id recensione** → necessario per rispondere
- **id utente** → autore della recensione
- **voto e testo** → contenuto della recensione
- **risposta** → eventuale risposta già presente

Questo permette al gestore di vedere immediatamente quali recensioni hanno già una risposta e quali no.

Apertura del form di risposta

Il metodo `apriFormRisposta()` gestisce l'intero flusso.

1. Selezione della recensione

Viene verificato che una riga sia selezionata.

2. Creazione della view

Viene creata una nuova `RispondiRecensioneView`.

3. Creazione del controller dedicato

Viene istanziato un `RispondiRecensioneController`, al quale vengono passati:

- la view
- la connessione
- la callback `this::loadRecensioni` per aggiornare la tabella dopo l'invio
- l'ID della recensione
- il testo della recensione

4. Apertura della finestra

Viene creato un nuovo `Stage` e mostrato il popup.

In pratica

La classe `RisposteGestoreController` funge da:

- **intermediario** tra GUI e server
- **gestore della tabella delle recensioni ricevute**
- **controller per l'apertura del form di risposta**
- **coordinatore della navigazione**

È una componente essenziale per permettere al gestore di mantenere un dialogo attivo con i clienti.

Conclusione

La classe `RisposteGestoreController` implementa l'intero flusso di gestione delle recensioni ricevute:

carica i dati dal server, li visualizza nella tabella e permette di rispondere tramite un popup dedicato.

È un controller fondamentale per la gestione della reputazione e della comunicazione con gli utenti.

RistoranteDettagliController

Funzione della classe

La classe **RistoranteDettagliController** gestisce l'intera schermata dei dettagli di un ristorante.

Coordina:

- il caricamento delle informazioni complete del ristorante
- la visualizzazione delle recensioni e delle eventuali risposte del gestore
- l'invio di nuove recensioni da parte del cliente
- l'aggiunta ai preferiti
- la gestione delle risposte del gestore (solo se proprietario)
- la navigazione verso la schermata precedente

È quindi il **controller centrale** per la consultazione e l'interazione con un singolo ristorante.

Analisi delle responsabilità principali:

Gestione della vista RistoranteDettagli

Il controller riceve:

- la view **RistoranteDettagliView**
- la connessione al server **ClientConnection**
- l'ID del ristorante da visualizzare
- la fonte del ristorante (UTENTE, GOOGLE, MICHELIN...)
- una callback **onBack** per tornare alla schermata precedente

Subito dopo la costruzione:

- inizializza gli handler
- controlla i permessi del cliente
- controlla i permessi del gestore
- carica i dettagli del ristorante

Inizializzazione degli handler

Il metodo `initHandlers()` collega i pulsanti della GUI:

- **Indietro** → ritorna alla schermata precedente
- **Aggiungi ai preferiti** → esegue `aggiungiPreferito()`
- **Scrivi recensione** → esegue `inviaRecensione()`

La logica rimane chiara e separata dalla view.

Controllo dei permessi del cliente

Il metodo `controllaPermessiCliente()` verifica se l'utente loggato è un cliente:

```
java
if (!UtenteDTO.getUtenteLoggato().isCliente()) {
    view.nascondiFunzioniCliente();
}
```

Se non lo è, vengono nascoste:

- area recensioni
- pulsante "Aggiungi ai preferiti"
- pulsante "Scrivi recensione"

Controllo dei permessi del gestore

Il metodo `controllaPermessiGestore()` verifica se il gestore loggato è **proprietario del ristorante**.

Invia al server:

- `idRistorante`
- `idGestore`

Se il server conferma, imposta:

```
java  
gestoreProprietario = true;
```

Questo abilita la possibilità di:

- rispondere alle recensioni
- vedere recensioni non anonime

Aggiunta ai preferiti

Il metodo `aggiungiPreferito()` invia al server:

- ID utente
- ID ristorante
- fonte del ristorante

La richiesta è:

```
java
```

MessageType.AGGIUNGI_PREFERITO

Se l'operazione va a buon fine, viene mostrato:

Aggiunto ai preferiti!

Invio di una recensione

Il metodo `inviaRecensione()`:

1. legge testo e voto dalla view
2. valida che il testo non sia vuoto
3. invia la richiesta `AGGIUNGI_RECENSIONE`
4. ricarica le recensioni aggiornate

Questo permette al cliente di lasciare feedback immediato.

Caricamento dei dettagli del ristorante

Il metodo `caricaDettagli()`:

1. invia una richiesta al server
 - `VISUALIZZA_UTENTE` se la fonte è `UTENTE`
 - `VISUALIZZA` altrimenti
2. aggiorna tutte le label della view
3. chiama `caricaRecensioni()` per mostrare i feedback

Vengono caricati:

- nome
- indirizzo
- città
- nazione
- tipo cucina
- fascia prezzo
- coordinate

- servizi disponibili

Caricamento delle recensioni

Il metodo `caricaRecensioni()` gestisce:

- recensioni
- risposte del gestore
- media voti

1. Scelta del tipo di richiesta

Se l'utente è loggato:

```
java  
VISUALIZZA_RECENSIONI_NON_ANONIME
```

Altrimenti:

```
java  
VISUALIZZA_RECENSIONI_ANONIME
```

2. Costruzione delle card

Per ogni recensione viene creata una card con:

- nome utente (o "Anonimo")
- voto
- testo
- data
- eventuale risposta del gestore

3. Risposta del gestore

Se:

- il gestore è proprietario
- la recensione non ha risposta

vengono mostrati:

- un `TextArea` per scrivere la risposta
- un pulsante “Rispondi”

che invia la risposta tramite `inviaRisposta()`.

4. Calcolo della media voti

Alla fine viene aggiornata la label:

Codice

Media voti: X.X / 5

oppure:

Codice

Nessuna recensione

Invio della risposta del gestore

Il metodo `inviaRisposta()`:

1. valida che il testo non sia vuoto

2. invia la richiesta **RISPONDI_RECENSIONE**
3. ricarica le recensioni aggiornate

Questo permette al gestore di rispondere direttamente dalla schermata dei dettagli.

In pratica

La classe **RistoranteDettagliController** funge da:

- **intermediario** tra GUI e server
- **gestore dei permessi cliente/gestore**
- **controller della visualizzazione completa del ristorante**
- **gestore delle recensioni e risposte**
- **coordinatore della navigazione**

È una delle classi più complesse e centrali dell'intero client.

Conclusione

La classe **RistoranteDettagliController** implementa l'intero flusso di consultazione e interazione con un ristorante:

carica i dettagli, mostra recensioni e risposte, gestisce preferiti, recensioni e risposte del gestore, e controlla i permessi in base al ruolo dell'utente.

È un controller fondamentale per l'esperienza utente e per la gestione della reputazione dei ristoranti.

RistoranteFormController

Funzione della classe

La classe **RistoranteFormController** gestisce l'intero flusso di creazione o modifica di un ristorante da parte del gestore.

Coordina:

- la lettura dei dati inseriti nel form
- la validazione dei campi obbligatori
- la costruzione della richiesta verso il server
- la distinzione tra aggiunta e modifica
- la chiusura della finestra e l'aggiornamento della schermata principale

È quindi il controller centrale per la gestione dei ristoranti lato gestore.

Analisi delle responsabilità principali

Gestione della vista RistoranteForm

Il controller riceve:

- la view **RistoranteFormView**
- la connessione al server **ClientConnection**
- una callback **onSuccess** da eseguire dopo il salvataggio
- l'ID del ristorante (null se si tratta di aggiunta)
- lo **Stage** della finestra da chiudere

Subito dopo la costruzione:

- inializza gli handler
- attende l'interazione dell'utente

Inizializzazione degli handler

Il metodo `initHandlers()` collega i pulsanti della GUI:

- **Salva** → esegue `salva()`
- **Annulla** → chiude la finestra e richiama `onSuccess`

La logica rimane chiara e separata dalla view.

Salvataggio dei dati

Il metodo `salva()` gestisce l'intero processo di aggiunta o modifica.

1. Lettura dei campi

Vengono letti:

- nome
- indirizzo
- tipo cucina
- città
- nazione
- fascia prezzo
- latitudine e longitudine
- servizi (delivery, prenotazione)

2. Validazione

Il controller verifica che:

- nome, indirizzo e tipo cucina non siano vuoti

- latitudine e longitudine siano numeri validi

In caso contrario, mostra un messaggio di errore.

3. Preparazione del payload JSON

Vengono inseriti tutti i campi nel JSON:

Codice

nome, indirizzo, tipo_cucina, fascia_prezzo,
citta, nazione, latitudine, longitudine,
delivery, prenotazione

4. Distinzione tra aggiunta e modifica

Se `idRistorante == null` → **aggiunta**:

Codice

`MessageType.AGGIUNGI_RISTORANTE`

Se `idRistorante != null` → **modifica**:

Codice

`MessageType.MODIFICA_RISTORANTE`

5. Invio della richiesta

Il controller invia la richiesta al server e, se l'operazione va a buon fine:

- esegue `onSuccess.run()` per aggiornare la tabella principale
- chiude la finestra (`stage.close()`)

In pratica

La classe **RistoranteFormController** funge da:

- intermediario tra GUI e server
- gestore della validazione dei dati
- controller per aggiunta e modifica dei ristoranti
- coordinatore della chiusura della finestra e aggiornamento della lista

È una componente essenziale per la gestione dei ristoranti lato gestore.

Conclusione

La classe **RistoranteFormController** implementa l'intero flusso di creazione e modifica dei ristoranti:

legge i dati dal form, li valida, invia la richiesta al server e aggiorna la schermata principale.

È un controller fondamentale per la gestione operativa dei ristoranti all'interno dell'applicazione.

ScriviRecensioneController

Funzione della classe

La classe **ScriviRecensioneController** gestisce l'intero flusso di inserimento di una nuova recensione da parte dell'utente.

Coordina:

- la lettura del voto e del commento inseriti nel form
- la validazione dei campi obbligatori
- la costruzione della richiesta verso il server
- la chiusura della finestra dopo l'invio
- l'aggiornamento della schermata precedente tramite callback

È quindi il controller dedicato alla creazione di nuove recensioni lato cliente.

Analisi delle responsabilità principali

Gestione della vista ScriviRecensione

Il controller riceve:

- la view **ScriviRecensioneView**
- la connessione al server **ClientConnection**
- l'ID del ristorante a cui associare la recensione
- una callback **onSuccess** da eseguire dopo l'invio

Subito dopo la costruzione:

- inizializza gli handler
- attende l'interazione dell'utente

Inizializzazione degli handler

Il metodo `initHandlers()` collega i pulsanti della GUI:

- **Invia** → esegue `inviaRecensione()`
- **Annulla** → chiude la finestra senza salvare

La logica rimane chiara e separata dalla view.

Invio della recensione

Il metodo `inviaRecensione()` gestisce l'intero processo di creazione della recensione.

1. Lettura dei campi

Vengono letti:

- voto selezionato
- commento inserito dall'utente

2. Validazione

Il controller verifica che:

- il voto non sia nullo
- il commento non sia vuoto

In caso contrario, mostra un messaggio di errore nella view.

3. Preparazione del payload JSON

Vengono inseriti nel JSON:

Codice
idRistorante
idUtente
voto
commento

4. Invio della richiesta

La richiesta inviata è:

Codice

MessageType.AGGIUNGI_RECENSIONE

Se il server risponde con successo:

- la finestra viene chiusa
- viene eseguita la callback `onSuccess` per aggiornare la schermata precedente

In caso di errore, viene mostrato il messaggio restituito dal server.

In pratica

La classe **ScriviRecensioneController** funge da:

- intermediario tra GUI e server
- validatore dei dati inseriti dall'utente
- controller dedicato alla creazione di nuove recensioni
- coordinatore della chiusura della finestra e aggiornamento della UI

È una componente essenziale per permettere agli utenti di lasciare feedback sui ristoranti.

Conclusione

La classe **ScriviRecensioneController** implementa l'intero flusso di inserimento di una recensione:

legge i dati dal form, li valida, invia la richiesta al server e aggiorna la schermata principale.

È un controller fondamentale per la partecipazione attiva degli utenti all'interno dell'applicazione.

Src → client → gui

RecensioniRistoranteView

Funzione della classe

La classe **RecensioniRistoranteView** rappresenta la schermata grafica dedicata alla visualizzazione delle recensioni ricevute dai ristoranti gestiti dall'utente.

È una view passiva: non contiene logica applicativa, ma fornisce al controller la struttura grafica necessaria per mostrare i dati provenienti dal server.

Serve come interfaccia di consultazione, permettendo al gestore di analizzare i feedback ricevuti.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce un layout composto da:

- un **titolo centrale** che identifica la schermata
- una **tabella JavaFX** configurata per espandersi e adattarsi allo spazio disponibile
- un **pulsante di chiusura** per tornare alla schermata precedente

La view non gestisce eventi: espone solo i componenti tramite getter.

Struttura grafica

- **Top (VBox)**: contiene il titolo, centrato e distanziato
- **Center (TableView)**: tabella con colonne configurate dal controller
- **Bottom (Button)**: pulsante "Chiudi"

Componenti principali

- `TableView<RecensioneRistoranteRow>` → visualizza le recensioni
- `Button btnChiudi` → chiude la finestra

In pratica

La classe fornisce un contenitore grafico semplice, pulito e facilmente aggiornabile dal controller tramite `Platform.runLater()`.

Conclusione

RecensioniRistoranteView è una view essenziale, progettata per mostrare dati senza logica interna.

Il controller si occupa di popolare la tabella e gestire la chiusura.

RecensioniUtenteView

Funzione della classe

La classe **RecensioniUtenteView** rappresenta la schermata dedicata alle recensioni scritte dall'utente.

Permette la consultazione, modifica ed eliminazione delle recensioni personali.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- un **titolo** che identifica la sezione
- una **tabella** con colonne per voto, testo, data, ecc.
- pulsanti per:
 - tornare indietro
 - modificare la recensione selezionata
 - eliminarla

Struttura grafica

- **Top**: titolo
- **Center**: tabella espandibile
- **Bottom**: pulsanti operativi

Componenti principali

- `TableView<RecensioneRow>` → mostra le recensioni dell'utente
- `Button btnModifica` → apre popup di modifica
- `Button btnElimina` → elimina la recensione
- `Button btnIndietro` → torna alla schermata precedente

In pratica

La view fornisce un'interfaccia completa per la gestione delle recensioni personali, lasciando al controller la logica di modifica ed eliminazione.

Conclusione

RecensioniUtenteView è una view strutturata e funzionale, pensata per operazioni CRUD sulle recensioni dell'utente.

RegisterView

Funzione della classe

La classe **RegisterView** rappresenta il form di registrazione dell'applicazione.

Permette all'utente di inserire i propri dati e scegliere il ruolo (cliente o gestore).

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- un **titolo**
- un **form centrale** con campi di testo e password
- una **checkbox** per selezionare il ruolo gestore
- pulsanti per registrarsi o tornare indietro

Struttura grafica

- **Top**: titolo
- **Center**: form con campi
- **Bottom**: pulsanti

Componenti principali

- `TextField txtUsername`
- `TextField txtEmail`
- `PasswordField txtPassword`
- `PasswordField txtConfermaPassword`
- `CheckBox chkGestore`
- `Button btnRegister`
- `Button btnIndietro`

In pratica

La view fornisce un'interfaccia chiara e ordinata per l'inserimento dei dati necessari alla registrazione.

Conclusione

RegisterView è una view semplice e funzionale, progettata per essere gestita dal RegisterController.

RicercaRistorantiView

Funzione della classe

La classe **RicercaRistorantiView** rappresenta la schermata di ricerca avanzata dei ristoranti.

Offre numerosi filtri, un campo di ricerca testuale e una tabella per visualizzare i risultati.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- una **barra superiore scrollabile** contenente tutti i filtri
- un **campo di ricerca testuale**
- filtri per:
 - tipo cucina
 - località
 - fascia di prezzo minima e massima
 - delivery
 - prenotazione
 - numero minimo di stelle
- una **tabella espandibile** per i risultati

Struttura grafica

- **Top**: barra filtri dentro uno ScrollPane
- **Center**: tabella dei risultati

Componenti principali

- `TextField txtRicerca`
- `ComboBox<String> filtroTipoCucina`
- `ComboBox<String> filtroLocalita`
- `ComboBox<Integer> filtroPrezzoMin`
- `ComboBox<Integer> filtroPrezzoMax`
- `CheckBox chkDelivery`
- `CheckBox chkPrenotazione`
- `ComboBox<Integer> filtroStelleMin`
- `Button btnCerca`
- `Button btnIndietro`
- `TableView<RistoranteRow> tabella`

In pratica

La view permette all'utente di impostare ricerche complesse e visualizzare i risultati in modo ordinato e responsive.

Conclusione

RicercaRistorantiView è una view ricca e flessibile, progettata per supportare ricerche avanzate lato client.

RispondiRecensioneView

Funzione della classe

La classe **RispondiRecensioneView** rappresenta la finestra dedicata alla risposta di un gestore a una recensione ricevuta.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- un **titolo** contenente il testo della recensione originale
- una **TextArea espandibile** per scrivere la risposta
- pulsanti per inviare o annullare

Struttura grafica

- **Center**: recensione + area di testo
- **Bottom**: pulsanti

Componenti principali

- `Label lblRecensione`
- `TextArea txtRisposta`
- `Button btnInvia`
- `Button btnAnnulla`

In pratica

La view fornisce un'interfaccia semplice e pulita per permettere al gestore di rispondere rapidamente.

Conclusione

RispondiRecensioneView è una view minimale e focalizzata, ideale per finestre popup di risposta.

RisposteGestoreView

Funzione della classe

La classe **RisposteGestoreView** rappresenta la schermata in cui il gestore visualizza tutte le recensioni ricevute e le eventuali risposte già inviate.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- un **titolo**
- una **tabella** con colonne per utente, voto, commento e risposta
- pulsanti per tornare indietro o rispondere

Struttura grafica

- **Top**: titolo
- **Center**: tabella espandibile
- **Bottom**: pulsanti

Componenti principali

- `TableView<RispostaRow> tabella`
- `Button btnIndietro`
- `Button btnRispondi`

In pratica

La view consente una consultazione chiara delle recensioni e delle risposte già fornite.

Conclusione

RisposteGestoreView è una view completa per la gestione delle interazioni tra gestore e clienti.

RistoranteDettagliView

Funzione della classe

La classe **RistoranteDettagliView** rappresenta la schermata dettagliata di un ristorante, con informazioni complete, recensioni e possibilità di lasciare un feedback.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- etichette per tutte le informazioni del ristorante
- un contenitore scrollabile per le recensioni
- un form per scrivere una nuova recensione
- pulsanti per preferiti, invio recensione e ritorno

Struttura grafica

- **Center**: ScrollPane con tutte le informazioni
- **VBox** interna con:
 - dati del ristorante
 - recensioni
 - form recensione

Componenti principali

- `Label lblNome, lblIndirizzo, lblCitta, ecc.`
- `VBox recensioniContainer`
- `TextArea txtRecensione`
- `Spinner<Integer> votoSpinner`
- `Button btnScriviRecensione`
- `Button btnPreferiti`

- Button `btnIndietro`

In pratica

La view permette una consultazione completa del ristorante e la partecipazione attiva tramite recensioni.

Conclusione

RistoranteDettagliView è una view ricca e centrale nell'esperienza dell'utente.

RistoranteFormView

Funzione della classe

La classe **RistoranteFormView** rappresenta il form di inserimento o modifica dei dati di un ristorante.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- un **titolo**
- un **form a griglia** con tutti i campi necessari
- checkbox per delivery e prenotazione
- pulsanti per salvare o annullare

Struttura grafica

- **Top**: titolo
- **Center**: GridPane con campi
- **Bottom**: pulsanti

Componenti principali

- `TextField txtNome, txtIndirizzo, txtTipoCucina, ecc.`
- `Spinner<Integer> spFasciaPrezzo`
- `CheckBox chkDelivery, chkPrenotazione`
- `Button btnSalva, btnAnnulla`

In pratica

La view fornisce un'interfaccia completa per la gestione dei ristoranti lato gestore.

Conclusione

RistoranteFormView è una view fondamentale per le operazioni CRUD dei ristoranti.

ScriviRecensioneView

Funzione della classe

La classe **ScriviRecensioneView** rappresenta la schermata dedicata alla creazione di una nuova recensione da parte dell'utente.

Analisi delle responsabilità principali

Gestione della vista

La classe costruisce:

- un **titolo**
- un **ComboBox** per il voto
- una **TextArea** per il commento
- un **messaggio di errore**
- pulsanti per inviare o annullare

Struttura grafica

- **Top**: titolo
- **Center**: form dentro uno ScrollPane
- **Bottom**: pulsanti

Componenti principali

- `ComboBox<Integer> cmbVoto`
- `TextArea txtCommento`
- `Label lblErrore`
- `Button btnInvia, btnAnnulla`

In pratica

La view permette all'utente di lasciare recensioni in modo semplice e guidato.

Conclusione

ScriviRecensioneView è una view essenziale per la partecipazione attiva dell'utente.

RecensioneRow

Rappresenta una recensione con:

- id
- id ristorante
- id utente
- voto
- testo
- data
- fonte

Utilizzata nelle tabelle delle recensioni.

RispostaRow

Rappresenta una recensione con eventuale risposta del gestore:

- id
- utente
- voto
- commento
- risposta

Utilizzata nella schermata RisposteGestore.

RistoranteRow

Rappresenta un ristorante nei risultati di ricerca o preferiti:

- id
- nome

- tipo cucina
- indirizzo
- fonte

Utilizzata nelle tabelle dei ristoranti.

Utente

Modello dati dell'utente:

- id
- nome
- email
- password
- ruolo

Utilizzato per gestire lo stato dell'utente loggato.

Src → client → controller

RecensioneDTO

Funzione della classe

La classe **RecensioneDTO** rappresenta il modello dati utilizzato per trasferire le informazioni relative a una recensione tra client e server.

È un **Data Transfer Object**, quindi non contiene logica applicativa, ma solo campi, costruttori e metodi di accesso.

Viene utilizzata in tutte le operazioni che coinvolgono recensioni, come:

- caricamento delle recensioni di un ristorante
- caricamento delle recensioni scritte dall'utente
- modifica di una recensione
- eliminazione
- visualizzazione nei controller dedicati

È quindi un elemento centrale per la gestione dei feedback degli utenti.

Analisi delle responsabilità principali

Rappresentazione dei dati della recensione

La classe incapsula tutti i campi necessari per descrivere una recensione:

- **id** → identificativo univoco
- **idUtente** → autore della recensione

- **idRistorante** → ristorante a cui si riferisce
- **voto** → valore numerico da 1 a 5
- **commento** → testo della recensione
- **data** → data in formato ISO (yyyy-MM-dd)

Questi campi vengono popolati dal server e utilizzati dal client per mostrare le recensioni nelle varie schermate.

Compatibilità con Gson

Il costruttore vuoto è necessario per permettere a Gson di deserializzare automaticamente gli oggetti JSON ricevuti dal server.

Utilizzo nei controller

La classe viene utilizzata da:

- **RecensioniUtenteController** → per mostrare, modificare ed eliminare recensioni personali
- **RecensioniRistoranteController** → per mostrare le recensioni ricevute dai ristoranti del gestore
- **RistoranteDettagliController** → per mostrare le recensioni nella scheda dettagliata

In tutti questi casi, il DTO rappresenta la struttura dati di base.

In pratica

RecensioneDTO funge da ponte tra:

- server (che invia i dati)
- controller (che li interpreta)
- view (che li visualizza)

È un contenitore semplice ma fondamentale per la gestione delle recensioni.

Conclusione

La classe RecensioneDTO fornisce una rappresentazione completa e coerente delle recensioni nel sistema.

È un componente essenziale per tutte le funzionalità legate ai feedback degli utenti.

RistoranteDTO

Funzione della classe

La classe **RistoranteDTO** rappresenta il modello dati completo di un ristorante, utilizzato per trasferire le informazioni tra client e server.

È un **Data Transfer Object** che contiene tutti i campi necessari per descrivere un ristorante in modo dettagliato.

Viene utilizzata in:

- ricerca ristoranti
- preferiti
- gestione ristoranti lato gestore
- visualizzazione dettagliata
- sincronizzazione con fonti esterne (THEKNIFE)

Analisi delle responsabilità principali

Rappresentazione completa del ristorante

La classe incapsula tutti i dati fondamentali:

- `id`
- `nome`
- `indirizzo`

- `citta`
- `nazione`
- `latitudine`
- `longitudine`
- `fasciaPrezzo`
- `tipoCucina`
- `delivery`
- `prenotazione`
- `fonte` → indica se il ristorante proviene da THEKNIFE o è stato creato dall'utente

Questa struttura permette al client di gestire ristoranti provenienti da fonti diverse.

Gestione della provenienza (fonte)

Il campo `fonte` è particolarmente importante perché permette di distinguere:

- ristoranti importati da THEKNIFE
- ristoranti creati manualmente dal gestore

La classe fornisce anche due metodi helper:

- `isTheKnife()`
- `isUtente()`

che semplificano la logica nei controller.

Compatibilità con Gson

Il costruttore vuoto permette la deserializzazione automatica dei dati JSON.

Utilizzo nei controller

RistoranteDTO viene utilizzato da:

- **RicercaRistorantiController** → per mostrare i risultati della ricerca
- **PreferitiController** → per mostrare i ristoranti salvati
- **GestoreRistorantiController** → per gestire CRUD dei ristoranti del gestore
- **RistoranteDettagliController** → per mostrare i dettagli completi

In tutti questi casi, il DTO rappresenta la fonte dati principale.

In pratica

La classe funge da modello dati universale per i ristoranti, indipendentemente dalla loro origine.

Permette al client di trattare in modo uniforme ristoranti provenienti da fonti diverse.

Conclusione

RistoranteDTO è un DTO completo e flessibile, progettato per rappresentare in modo accurato i ristoranti nel sistema.

È uno dei modelli più importanti dell'intera applicazione.

UtenteDTO

Funzione della classe

La classe **UtenteDTO** rappresenta l'utente loggato all'interno dell'applicazione.

È un **Data Transfer Object** che funge anche da **Singleton**, mantenendo un'unica istanza globale accessibile da tutti i controller.

Viene utilizzata per:

- memorizzare i dati dell'utente autenticato
- determinare il ruolo (CLIENTE, GESTORE, ADMIN)
- controllare la visibilità dinamica della GUI
- inviare richieste al server con l'ID utente

Analisi delle responsabilità principali

Gestione dello stato utente tramite Singleton

La classe implementa un Singleton tramite:

- `creaUtenteLoggato(...)` → crea o sostituisce l'istanza
- `getUtenteLoggato()` → restituisce l'utente corrente

Questo permette a qualsiasi controller di accedere ai dati dell'utente senza passaggi espliciti.

Rappresentazione dei dati dell'utente

La classe contiene:

- `id`
- `username`
- `email`
- `ruolo`

Questi campi vengono popolati al momento del login e rimangono disponibili per tutta la sessione.

Determinazione del ruolo

La classe fornisce metodi helper:

- `isGestore()`
- `isCliente()`

Questi metodi sono fondamentali per:

- mostrare/nascondere pulsanti
- abilitare funzionalità specifiche
- controllare i permessi lato client

Utilizzo nei controller

UtenteDTO è utilizzato da:

- **HomeController** → per aggiornare la visibilità dei pulsanti
- **LoginController** → per salvare l'utente loggato
- **RegisterController** → dopo una registrazione
- **PreferitiController**, **RecensioniController**, ecc. → per inviare l'ID utente al server

In pratica

La classe funge da:

- memoria centrale dell'utente loggato
- fonte di verità per ruolo e permessi

- meccanismo di controllo della UI
- supporto per tutte le richieste al server

Conclusione

UtenteDTO è un modello dati fondamentale per l'intero client.

Gestisce lo stato dell'utente, i permessi e la visibilità dinamica delle schermate, garantendo coerenza tra tutte le sezioni dell'applicazione.

Src → client → net

ClientConnection

Funzione della classe

La classe **ClientConnection** gestisce l'intera comunicazione di rete tra il client e il server.

È responsabile dell'apertura della connessione TCP, dell'invio delle richieste e della ricezione delle risposte.

Rappresenta quindi il **livello di comunicazione basso** dell'applicazione, su cui si appoggiano tutti i controller.

Analisi delle responsabilità principali

Gestione della connessione TCP

Nel costruttore viene aperta una connessione verso il server:

- **Socket** → canale di comunicazione
- **PrintWriter** → invio dei dati
- **BufferedReader** → ricezione dei dati

La connessione rimane aperta per tutta la durata dell'applicazione.

Invio delle richieste

Il metodo:

Codice

```
sendRequest(Request req)
```

esegue l'intero flusso:

1. converte la richiesta in JSON
2. la invia al server tramite `out.println()`
3. attende una risposta JSON
4. la converte in un oggetto `Response`

Questo metodo è utilizzato da tutti i controller per comunicare con il server.

Chiusura della connessione

Il metodo `close()` chiude il socket e libera le risorse.

In pratica

ClientConnection funge da:

- ponte tra client e server
- gestore del protocollo di comunicazione
- componente centrale per tutte le operazioni di rete

Senza questa classe, nessun controller potrebbe inviare richieste o ricevere dati.

Conclusione

ClientConnection è la base dell'intero sistema di comunicazione.

Fornisce un'interfaccia semplice e affidabile per inviare richieste e ricevere risposte dal server.

Request

Funzione della classe

La classe **Request** rappresenta una richiesta inviata dal client al server.

Incapsula:

- il tipo di messaggio (**MessageType**)
- il payload JSON contenente i parametri

Viene utilizzata da tutti i controller per costruire richieste standardizzate.

Analisi delle responsabilità principali

Rappresentazione della richiesta

Una richiesta è composta da:

- **type** → identifica l'operazione (LOGIN, CERCA_RISTORANTI, ELIMINA_RISTORANTE, ecc.)
- **payload** → contiene i parametri necessari all'operazione

Questo permette al server di interpretare correttamente la richiesta.

Serializzazione in JSON

Il metodo:

Codice
toJson()

costruisce un oggetto JSON con:

- "type" → nome del messaggio
- "payload" → oggetto JSON dei parametri

Il risultato è una stringa JSON inviata tramite `ClientConnection`.

Compatibilità con Gson

La classe utilizza `JsonObject` per garantire compatibilità diretta con Gson e con il formato delle risposte.

In pratica

Request è utilizzata da tutti i controller per:

- inviare parametri al server
- specificare il tipo di operazione
- mantenere un formato uniforme

Conclusione

Request è una classe semplice ma fondamentale:

standardizza la comunicazione e garantisce che ogni richiesta sia strutturata correttamente.

Response

Funzione della classe

La classe **Response** rappresenta la risposta ricevuta dal server dopo l'invio di una richiesta.

Contiene:

- il tipo di messaggio
- il payload dei dati
- un flag di successo
- un eventuale messaggio di errore

È il principale mezzo con cui il server comunica i risultati delle operazioni.

Analisi delle responsabilità principali

Parsing della risposta JSON

Il metodo statico:

Codice
`fromJson(String json)`

utilizza Gson per convertire la stringa JSON in un oggetto Response.

Questo permette ai controller di lavorare direttamente con oggetti Java.

Accesso ai dati

La classe fornisce getter per:

- `type` → tipo della risposta
- `data` → payload JSON
- `success` → esito dell'operazione
- `message` → eventuale messaggio informativo o di errore

Compatibilità con versioni precedenti

Sono presenti alias come:

- `isOk()` → equivalente di `isSuccess()`
- `getPayload()` → equivalente di `getData()`

Questo garantisce compatibilità con codice già esistente.

Utilizzo nei controller

Response viene utilizzata per:

- verificare se l'operazione è andata a buon fine
- estrarre dati JSON (ristoranti, recensioni, preferiti, ecc.)
- mostrare messaggi di errore
- aggiornare la GUI

In pratica

Response funge da:

- contenitore dei dati restituiti dal server
- indicatore dell'esito dell'operazione
- interfaccia di comunicazione per tutti i controller

Conclusione

Response è una classe essenziale per interpretare correttamente le risposte del server.

Permette ai controller di gestire facilmente sia i dati ricevuti sia gli eventuali errori.

Src → client → common

MessageType

Funzione della classe

`MessageType` è un **enum centrale** che definisce *tutti* i tipi di messaggi che possono essere scambiati tra client e server.

Rappresenta il **protocollo logico** dell'applicazione: ogni operazione possibile è identificata da un valore univoco.

È utilizzato sia dal client (per inviare richieste) sia dal server (per interpretarle e rispondere).

Analisi delle responsabilità principali

Identificazione univoca delle operazioni

Ogni valore dell'enum rappresenta un'azione specifica, ad esempio:

- LOGIN, REGISTRAZIONE
- CERCA_RISTORANTI, VISUALIZZA
- AGGIUNGI_RISTORANTE, MODIFICA_RISTORANTE, ELIMINA_RISTORANTE
- AGGIUNGI_RECENSIONE, MODIFICA_RECENSIONE, ELIMINA_RECENSIONE
- AGGIUNGI_PREFERITO, RIMUOVI_PREFERITO, VISUALIZZA_PREFERITI

Questo permette al server di sapere esattamente quale operazione deve eseguire.

Separazione logica per categorie

L'enum è organizzato in blocchi:

- **UTENTE** → login, registrazione
- **RISTORANTI MICHELIN** → operazioni in sola lettura
- **RISTORANTI UTENTE (CRUD)** → operazioni del gestore
- **RECENSIONI** → gestione completa delle recensioni
- **PREFERITI** → gestione dei preferiti

Questa struttura rende il protocollo leggibile e scalabile.

Coerenza tra client e server

Il valore dell'enum viene serializzato come stringa e inviato nel campo **"type"** della richiesta JSON.

Il server lo deserializza e lo usa per instradare la richiesta verso il metodo corretto.

In pratica

MessageType è la **mappa delle operazioni** dell'intero sistema.

Senza di esso, client e server non avrebbero un linguaggio comune.

Conclusione

È uno dei componenti più importanti del protocollo: definisce tutte le azioni possibili e garantisce coerenza tra client e server.

common.Request

Funzione della classe

La classe `common.Request` rappresenta una **richiesta generica** inviata dal client al server.

È la versione “server-side” della Request del client, con struttura identica ma senza metodi di serializzazione.

Serve principalmente al server per:

- ricevere la richiesta JSON
- deserializzarla tramite Gson
- accedere al tipo (`MessageType`)
- accedere al payload (`JsonObject`)

Analisi delle responsabilità principali

1. Rappresentazione dei dati della richiesta

La classe contiene due campi pubblici:

- `MessageType type` → identifica l'operazione richiesta
- `JsonObject payload` → contiene i parametri necessari

Questa struttura è semplice e diretta, perfetta per Gson.

Compatibilità con Gson

Il costruttore vuoto è fondamentale:

Gson può creare automaticamente un oggetto `Request` a partire dal JSON ricevuto.

Utilizzo lato server

Il server, dopo aver ricevuto la stringa JSON:

1. la converte in `Request`
2. legge `type`
3. in base al tipo, esegue l'operazione corretta
4. usa `payload` per ottenere i parametri

In pratica

`common.Request` è la **forma deserializzata** della richiesta del client.

È il punto di ingresso di ogni operazione lato server.

Conclusione

Una classe semplice ma essenziale: permette al server di interpretare correttamente le richieste provenienti dal client.

common.Response

Funzione della classe

La classe `common.Response` rappresenta la **risposta generica** che il server invia al client.

Contiene:

- il tipo della risposta
- un flag di successo
- un messaggio
- un payload dati

È la controparte server-side della `client.net.Response`.

Analisi delle responsabilità principali

Struttura della risposta

La classe contiene:

- `String type` → tipo della risposta (spesso uguale alla richiesta)
- `boolean success` → indica se l'operazione è riuscita
- `String message` → messaggio informativo o di errore
- `JsonObject data` → dati restituiti al client

Metodi statici factory

La classe fornisce tre metodi statici molto utili:

`ok()`

Crea una risposta positiva senza dati.

`ok(JsonObject data)`

Crea una risposta positiva con dati.

`error(String msg)`

Crea una risposta negativa con messaggio di errore.

Questi metodi:

- semplificano il codice del server
- garantiscono risposte uniformi
- evitano duplicazione di codice

Compatibilità con Gson

La classe è progettata per essere serializzata automaticamente da Gson e inviata al client come JSON.

Utilizzo lato server

Ogni handler del server restituisce un oggetto `Response`, che viene poi convertito in JSON e inviato al client.

In pratica

`common.Response` è il **formato standard** con cui il server comunica l'esito delle operazioni al client.

Conclusione

È una classe fondamentale per garantire risposte coerenti, leggibili e facilmente interpretabili dal client.

Src → client

ClientMain

Funzione della classe

La classe **ClientMain** è il punto di ingresso dell'applicazione JavaFX lato client.

Estende `Application` e ha tre responsabilità principali:

- inizializzare la connessione di rete verso il server (`ClientConnection`)
- creare e mostrare la prima schermata (Home o Login, a seconda del flusso)
- fungere da “**router**” **centrale** tra le varie schermate (Home, Login, Registrazione, Ricerca, Dettagli, Preferiti, Gestione, Recensioni, ecc.)

In pratica è il **coordinatore dell'interfaccia grafica**: decide quale view mostrare e con quale controller, mantenendo un unico `Stage` principale.

Analisi delle responsabilità principali

Avvio dell'applicazione e connessione al server

Nel metodo `start(Stage stage)`:

- salva il riferimento allo **Stage** principale (`primaryStage`)
- prova a creare una nuova `ClientConnection("localhost", 5555)`
- in caso di successo, stampa un messaggio di conferma
- in caso di errore, stampa un messaggio e termina senza avviare la GUI
- chiama `mostraHome()` come prima schermata

Questa fase è critica perché:

- se la connessione fallisce, l'app non procede
- se la connessione va a buon fine, tutti i controller useranno **la stessa** `ClientConnection` condivisa

Gestione centralizzata delle schermate

La classe definisce una serie di metodi privati, ognuno dedicato a una schermata:

- `mostraLogin()`
- `mostraRegistrazione()`
- `mostraHome()`
- `mostraRicercaRistoranti()`
- `mostraDettagliRistorante(int id, String fonte)`
- `mostraPreferiti()`
- `mostraRecensioniUtente()`
- `mostraRecensioniRicevute()`
- `mostraGestioneRistoranti()`

Ogni metodo segue sempre lo stesso pattern:

1. crea la **View** corrispondente
2. crea il **Controller**, passandogli:
 - la view
 - la `ClientConnection`
 - una o più **callback** (metodi `this::...`) per la navigazione
3. crea una nuova `Scene` con la view
4. la imposta sul `primaryStage`
5. imposta titolo e **schermo intero** (`setMaximized(true)`)

6. chiama `show()`

In questo modo, **tutta la navigazione** è centralizzata in un unico punto, e i controller non devono conoscere lo `Stage`: si limitano a chiamare le callback.

Uso delle callback per la navigazione

Un aspetto importante è l'uso sistematico di **lambda e method reference** per la navigazione.

Esempi:

in `mostraLogin()`:

```
java
new LoginController(
    view,
    connection,
    this::mostraHome,
    this::mostraRegistrazione,
    this::mostraHome
);
```

- Il controller non sa nulla di `Stage` o `Scene`: quando deve andare in Home o in Registrazione, chiama semplicemente la callback.

in `mostraHome()`:

```
java
new HomeController(
    view,
    connection,
    this::mostraRicercaRistoranti,
    this::mostraPreferiti,
    this::mostraRecensioniUtente,
    this::mostraGestioneRistoranti,
    this::mostraRecensioniRicevute,
    this::mostraLogin,
```

```
() -> {  
    System.out.println("Logout eseguito");  
    mostraHome();  
}  
);
```

- Qui la Home riceve tutte le “strade” possibili:
 - ricerca ristoranti
 - preferiti
 - recensioni utente
 - gestione ristoranti
 - recensioni ricevute
 - login
 - logout (che in pratica resetta lo stato e richiama `mostraHome()`)

Questo approccio ha due vantaggi enormi:

- **disaccoppia** i controller dalla gestione dello **Stage**
- rende la navigazione **esplicita e centralizzata** in ClientMain

Schermo intero per tutte le schermate principali

In ogni metodo di cambio schermata, dopo aver impostato la **Scene**, viene chiamato:

```
java  
primaryStage.setMaximized(true);
```

Questo garantisce che:

- Login
- Registrazione
- Home
- Ricerca
- Dettagli
- Preferiti
- Recensioni
- Gestione ristoranti

vengano tutte mostrate **a schermo intero**, mantenendo un'esperienza coerente.

Flussi specifici

Login

- `mostraLogin()` crea `LoginView` e `LoginController`
- il controller riceve:
 - callback per andare in Home dopo login riuscito
 - callback per andare in Registrazione
 - callback per tornare in Home (o schermata precedente) in caso di annullamento

Registrazione

- `mostraRegistrazione()` crea `RegisterView` e `RegisterController`
- il controller riceve:
 - callback per tornare al Login dopo registrazione
 - callback per tornare al Login in caso di annullamento

Home

- `mostraHome()` crea `HomeView` e `HomeController`
- il controller riceve tutte le callback per:
 - ricerca ristoranti
 - preferiti
 - recensioni utente
 - gestione ristoranti (gestore)
 - recensioni ricevute (gestore)
 - login
 - logout (lambda che stampa un messaggio e richiama `mostraHome()`)

Ricerca ristoranti

- `mostraRicercaRistoranti()` crea `RicercaRistorantiView` e `RicercaRistorantiController`
- il controller riceve:
 - callback per tornare in Home
 - callback `(id, fonte) -> mostraDettagliRistorante(id, fonte)` per aprire i dettagli

Dettagli ristorante

- `mostraDettagliRistorante(int id, String fonte)` crea `RistoranteDettagliView` e `RistoranteDettagliController`

- il controller riceve:
 - l'ID del ristorante
 - la fonte (THEKNIFE/UTENTE)
 - callback per tornare in Home

Preferiti

- `mostraPreferiti()` crea `PreferitiView` e `PreferitiController`
- il controller riceve:
 - callback per tornare in Home
 - callback per aprire i dettagli di un ristorante selezionato

Recensioni utente

- `mostraRecensioniUtente()` crea `RecensioniUtenteView` e `RecensioniUtenteController`
- il controller riceve:
 - callback per tornare in Home

Recensioni ricevute (gestore)

- `mostraRecensioniRicevute()` crea `RecensioniRistoranteView` e `RecensioniRistoranteController`

subito dopo la creazione del controller, viene chiamato:

java

```
controller.loadRecensioni();
```

- quindi la schermata viene mostrata già popolata.
- il controller riceve:
 - callback per tornare in Home

Gestione ristoranti (gestore)

- `mostraGestioneRistoranti()` crea `GestoreRistorantiView` e `GestoreRistorantiController`

subito dopo, viene chiamato:

java

```
controller.loadRiepilogo();
```

- per caricare i dati iniziali.
- il controller riceve:
 - callback per tornare in Home
 - callback per aprire i dettagli di un ristorante

In pratica

La classe **ClientMain** è:

- il **punto di ingresso** dell'applicazione
- il **gestore della connessione** verso il server
- il **router centrale** di tutte le schermate
- il **coordinatore** tra view e controller tramite callback

Tutti i controller dipendono da lei per la navigazione, ma non conoscono direttamente lo **Stage**: questo rende l'architettura più pulita e modulare.

Conclusione

ClientMain è il cuore dell'applicazione lato client:

- inizializza la connessione
- avvia la GUI
- gestisce la navigazione tra tutte le schermate
- mantiene coerenza di stile (schermo intero, titoli, flussi)

È la classe che tiene insieme **rete**, **controller** e **view** in un unico flusso coerente.

Src —> server —> connection

ClientHandler

Funzione della classe

`ClientHandler` rappresenta il **gestore dedicato a un singolo client**.

Ogni volta che un client si connette al server, viene creato un nuovo `ClientHandler` che:

- legge le richieste del client
- le deserializza
- le inoltra ai vari service
- invia la risposta JSON al client

È quindi il **cuore del protocollo server-side**.

Analisi delle responsabilità principali

Gestione della connessione

Nel metodo `run()`:

- apre gli stream `BufferedReader` e `PrintWriter`
- entra in un ciclo infinito
- legge ogni riga inviata dal client
- la interpreta come JSON
- la converte in `Request` tramite Gson
- invia la risposta convertita in JSON

Il ciclo termina solo quando il client chiude la connessione.

Parsing e validazione della richiesta

Ogni riga ricevuta viene convertita in:

```
java
Request req = gson.fromJson(line, Request.class);
```

Se la richiesta è nulla o senza tipo, viene restituito:

```
java
Response.error("Richiesta non valida")
```

Smistamento tramite switch

Il metodo `process(Request req)` contiene uno **switch centrale** basato su `req.type`.

Ogni case:

- chiama il service appropriato
- passa la richiesta
- restituisce una `Response`

Esempi:

- `LOGIN` → `UtenteService.login(req)`
- `CERCA_RISTORANTI` → `RistoranteService.cerca(req)`
- `AGGIUNGI_RECENSIONE` → `RecensioneService.aggiungi(req)`
- `AGGIUNGI_PREFERITO` → logica diretta + DAO

Questo switch è il **router principale del server**.

Gestione dei preferiti direttamente nel handler

Due operazioni (`AGGIUNGI_PREFERITO` e `RIMUOVI_PREFERITO`) sono gestite direttamente qui, senza passare da un service:

- estrazione parametri dal payload
- chiamata al DAO
- risposta immediata

Robustezza

Il blocco try/catch nel `run()` garantisce che eventuali errori non blocchino il server.

In pratica

`ClientHandler` è:

- il **dispatcher** delle richieste
- il **traduttore JSON** ↔ **oggetti Java**
- il **punto di contatto** tra client e service

È uno dei componenti più importanti dell'intero server.

ServerListener

Funzione della classe

ServerListener è il **thread principale del server**, responsabile di:

- aprire la porta TCP
- accettare nuove connessioni
- creare un `ClientHandler` per ogni client
- delegare l'esecuzione al thread pool

È il **gateway** del server.

Analisi delle responsabilità principali

Apertura della porta

Nel metodo `run()`:

```
java
```

```
ServerSocket server = new ServerSocket(port);
```

Il server rimane in ascolto sulla porta specificata.

Accettazione dei client

Il ciclo:

```
java
while (true) {
    Socket client = server.accept();
}
```

accetta ogni nuova connessione e stampa:

Codice

Nuovo client: <indirizzo>

Delegazione al thread pool

Per ogni client:

```
java
ClientHandler handler = new ClientHandler(client);
ServerThreadPool.submit(handler);
```

Questo garantisce:

- nessun blocco del thread principale
- gestione concorrente di più client
- scalabilità

Robustezza

Il try-with-resources garantisce la chiusura del ServerSocket in caso di errori.

In pratica

`ServerListener` è il **punto di ingresso** del server.

Accetta connessioni e le distribuisce al thread pool.

`ServerThreadPool`

Funzione della classe

`ServerThreadPool` gestisce il **pool di thread** utilizzato per eseguire i `ClientHandler`.

È un wrapper statico attorno a un `ExecutorService` configurato con 20 thread fissi.

Serve per:

- gestire più client contemporaneamente
- evitare la creazione continua di nuovi thread
- monitorare lo stato del pool

Analisi delle responsabilità principali

1. Pool fisso da 20 thread

```
java  
Executors.newFixedThreadPool(20)
```

Questo garantisce:

- massimo 20 client gestiti in parallelo
- nessun sovraccarico di creazione thread
- comportamento prevedibile

2. Invio dei task

```
java  
public static void submit(Runnable task)
```

Usato da `ServerListener` per eseguire ogni `ClientHandler`.

3. Monitoraggio

Metodi utili:

- `getActiveCount()` → thread attivi
- `getQueueSize()` → richieste in attesa

Questi sono utili per debug e monitoraggio del carico.

4. Shutdown

Supporta:

- `shutdown()` → chiusura gentile
- `shutdownNow()` → chiusura immediata
- `awaitTermination()` → attesa della chiusura

In pratica

`ServerThreadPool` è il **motore di esecuzione** del server.

Permette di gestire molti client in modo efficiente e stabile.

Src → server → dao

PreferitiDAO

PreferitiDAO gestisce l'intero sistema dei ristoranti preferiti degli utenti.

È un DAO particolare perché deve integrare due fonti dati distinte:

- ristoranti ufficiali THEKNIFE
- ristoranti creati dagli utenti

Per questo motivo ogni preferito memorizza anche la **fonte**, così il sistema sa da quale tabella recuperare i dati.

Funzioni principali

Aggiunta di un preferito

Il metodo `aggiungi()` inserisce un nuovo preferito nella tabella `preferiti`, salvando:

- id utente
- id ristorante
- fonte (THEKNIFE o UTENTE)

Restituisce `true` se l'inserimento è avvenuto.

Rimozione di un preferito

Il metodo `rimuovi()` elimina il preferito corrispondente.

Restituisce `true` se almeno una riga è stata cancellata.

Caricamento della lista completa dei preferiti

Il metodo `getRistorantiPreferiti()` è il cuore del DAO.

Per ogni preferito:

- legge `id_ristorante` e `fonte`
- se la `fonte` è `THEKNIFE` → carica dalla tabella `ristorantitheknife`
- se la `fonte` è `UTENTE` → carica dalla tabella `ristorantiutente`
- imposta la `fonte` nel modello `Ristorante` tramite `setFonte()`

Caricamento ristorante THEKNIFE

`getIdTheKnife()` esegue una query sulla tabella dei ristoranti ufficiali e restituisce un modello `Ristorante` completo.

Caricamento ristorante UTENTE

`getIdUtente()` esegue una query sulla tabella dei ristoranti creati dagli utenti e restituisce un modello `Ristorante` con `fonte` `UTENTE`.

Ruolo nel sistema

PreferitiDAO è il ponte tra due mondi diversi del database e permette di costruire una lista preferiti mista, coerente e uniforme.

RecensioneDAO

RecensioneDAO gestisce le recensioni e le risposte alle recensioni, con particolare attenzione ai ristoranti gestiti da un gestore.

Funzioni principali

Recupero recensioni dei ristoranti del gestore

Il metodo **getByGestore()**:

- unisce **recensioni** con **ristorantiutente**
- filtra solo recensioni con fonte UTENTE
- ordina per data decrescente
- restituisce una lista di **Recensione** completa di:
 - id recensione
 - id utente
 - id ristorante
 - voto
 - testo
 - data
 - fonte
 - nome del ristorante

È il metodo utilizzato dalla dashboard del gestore.

Risposta a una recensione

Il metodo `rispondi()` inserisce una risposta nella tabella `risposte`.

Non modifica la recensione originale, ma crea un record collegato tramite `id_recensione`.

Ruolo nel sistema

`RecensioneDAO` è il punto di accesso ai dati delle recensioni e delle risposte, utilizzato da `RecensioneService` e `ClientHandler`.

RistoranteDAO

`RistoranteDAO` gestisce i ristoranti ufficiali provenienti dalla fonte THEKNIFE.

È uno dei DAO più complessi perché implementa una ricerca avanzata con molti filtri.

Funzioni principali

Ricerca avanzata dei ristoranti THEKNIFE

Il metodo `cerca()` costruisce dinamicamente una query SQL con filtri su:

- nome
- tipo cucina
- città o nazione
- fascia prezzo
- delivery
- prenotazione
- stelle minime (HAVING su media recensioni)

Utilizza:

- `StringBuilder` per costruire condizioni dinamiche
- `List<Object>` per i parametri
- `LEFT JOIN` con `Recensioni` per calcolare la media dei voti

Restituisce una lista di `Ristorante` con fonte THEKNIFE.

Verifica esistenza ristorante utente

`esisteUtente()` controlla se un ristorante utente esiste nella tabella `ristorantiutente`.

Caricamento ristorante THEKNIFE tramite ID

`getId()` restituisce un oggetto `Ristorante` completo, con fonte THEKNIFE.

Ruolo nel sistema

`RistoranteDAO` è il motore di ricerca dei ristoranti ufficiali e viene utilizzato da `RistoranteService`, `PreferitiDAO` e `ClientHandler`.

RistorantiUtenteDAO

Questo DAO gestisce i ristoranti creati dagli utenti.

Il frammento che hai inviato riguarda la parte finale del file, ma è sufficiente per descriverlo.

Funzioni principali

Caricamento lista ristoranti utente

Il metodo `getAll()` (presente nella versione completa) restituisce tutti i ristoranti creati dagli utenti.

Caricamento ristorante UTENTE tramite ID

Il metodo `getId()`:

- esegue una query sulla tabella `RistorantiUtente`
- costruisce un modello `Ristorante`
- imposta la fonte a UTENTE

Ruolo nel sistema

RistorantiUtenteDAO è il punto di accesso ai ristoranti creati dagli utenti ed è utilizzato da **RistoranteService**, **PreferitiDAO** e **RecensioneDAO**.

UtenteDAO

UtenteDAO gestisce tutte le operazioni relative agli utenti: autenticazione, registrazione e ricerca.

Funzioni principali

Login

Il metodo **login()** verifica email e password.

Se le credenziali sono corrette, restituisce un oggetto **Utente**.

Registrazione

Il metodo **registrati()** inserisce un nuovo utente nella tabella **utenti**, salvando:

- nome
- email
- password
- ruolo

Ricerca per email

Il metodo **getByEmail()** restituisce l'utente associato a una determinata email.

Ruolo nel sistema

UtenteDAO è il punto di accesso ai dati degli utenti ed è utilizzato da UtenteService e ClientHandler.

Src → server → model

Recensione

Recensione è il modello dati che rappresenta una recensione lasciata da un utente su un ristorante.

La classe è progettata per gestire sia recensioni anonime sia recensioni con autore visibile, ed è costruita per integrarsi perfettamente con le query JOIN dei DAO.

Struttura e significato dei campi

id

- Identificativo univoco della recensione nel database.

idUtente

Identifica l'utente che ha scritto la recensione.

- Può essere usato per recuperare informazioni aggiuntive (nome, email, ruolo).

idRistorante

Identifica il ristorante recensito.

- Permette di collegare la recensione al ristorante corretto.

voto

Punteggio numerico assegnato dall'utente (tipicamente 1–5).

- È utilizzato anche per calcolare la media delle recensioni.

testo

- Contenuto della recensione.

data

- Data di pubblicazione della recensione, salvata come stringa formattata.

fonte

- Indica se la recensione proviene da:
 - THEKNIFE (ristoranti ufficiali)
 - UTENTE (ristoranti creati dagli utenti)

nomeRistorante

Nome del ristorante, ottenuto tramite JOIN nel DAO.

- Permette di mostrare la recensione senza dover ricaricare il ristorante.

nomeUtente

Nome dell'autore della recensione.

- Può essere `null` per recensioni anonime.

Costruttori

La classe fornisce due costruttori distinti:

Costruttore per recensioni anonime

- Imposta `nomeUtente = null`.

Costruttore per recensioni con autore visibile

- Accetta anche `nomeUtente`.

Questa distinzione permette al server di gestire entrambe le modalità senza duplicare logica.

Metodi

La classe espone getter per tutti i campi e setter per i campi modificabili (`voto`, `testo`, `data`, `fonte`, `nomeRistorante`, `nomeUtente`).

Ruolo nel sistema

`Recensione` è utilizzata da:

- `RecensioneDAO` per costruire oggetti recensione dalle query SQL
- `RistoranteDAO` per calcolare la media dei voti
- `ClientHandler` per inviare recensioni al client
- interfacce gestore e utente per mostrare recensioni e risposte

Risposta

Risposta rappresenta la risposta di un gestore a una recensione lasciata da un utente.

È un modello semplice ma fondamentale per completare il flusso di interazione tra cliente e gestore.

Struttura e significato dei campi

id

- Identificativo univoco della risposta.

idRecensione

Identifica la recensione a cui la risposta è collegata.

- Permette di mantenere la relazione $1 \rightarrow 1$ tra recensione e risposta.

testo

- Contenuto della risposta del gestore.

Costruttore

Il costruttore inizializza tutti i campi necessari per rappresentare una risposta completa.

Metodi

Getter per tutti i campi e setter per modificare il testo della risposta.

Ruolo nel sistema

Risposta è utilizzata da:

- **RecensioneDAO** per inserire nuove risposte
- **ClientHandler** per inviare le risposte al client
- interfacce gestore per mostrare le risposte alle recensioni

Ristorante

Ristorante è il modello dati principale che rappresenta un ristorante, indipendentemente dalla sua origine.

La classe è progettata per essere compatibile sia con i ristoranti ufficiali THEKNIFE sia con quelli creati dagli utenti.

Struttura e significato dei campi

id

- Identificativo univoco del ristorante.

nome

- Nome commerciale del ristorante.

indirizzo

- Indirizzo completo.

citta

- Città in cui si trova il ristorante.

nazione

- Nazione del ristorante.

latitudine / longitudine

- Coordinate geografiche per mappe e geolocalizzazione.

fasciaPrezzo

- Valore numerico (1–5) che rappresenta la fascia di prezzo.

tipoCucina

- Categoria culinaria (es. "Italiano", "Giapponese", "Messicano").

delivery

- Indica se il ristorante offre consegna a domicilio.

prenotazione

- Indica se il ristorante accetta prenotazioni.

fonte

- Indica la provenienza del ristorante:
 - "THEKNIFE" → ristorante ufficiale
 - "UTENTE" → ristorante creato da un utente

Costruttore

Il costruttore completo inizializza tutti i campi, inclusa la fonte.

Questo permette al server di distinguere immediatamente la provenienza del ristorante.

Metodi

Getter e setter per tutti i campi.

Metodo aggiuntivo importante:

```
java  
public boolean isTheKnife()
```

Restituisce **true** se il ristorante proviene dalla fonte THEKNIFE.

È utilizzato da DAO, Service e ClientHandler per applicare logiche diverse.

Ruolo nel sistema

Ristorante è utilizzato da:

- **RistoranteDAO**
- **RistorantiUtenteDAO**
- **PreferitiDAO**
- **RecensioneDAO** (per nome ristorante)
- **ClientHandler** per inviare i dati al client

Utente

Utente rappresenta un utente registrato nel sistema, sia esso cliente o gestore.

È un modello semplice ma centrale per autenticazione, autorizzazione e gestione delle recensioni.

Struttura e significato dei campi

id

- Identificativo univoco dell'utente.

nome

- Nome dell'utente, mostrato nelle recensioni non anonime.

email

- Email utilizzata per login e comunicazioni.

password

- Password dell'utente (gestita dal DB).

ruolo

- Può essere:
 - "utente"
 - "gestore"

Costruttore

Inizializza tutti i campi necessari per rappresentare un utente completo.

Metodi

Getter per tutti i campi e setter per i campi modificabili (`nome`, `email`, `password`, `ruolo`).

Ruolo nel sistema

`Utente` è utilizzato da:

- `UtenteDAO` per login, registrazione e ricerca
- `Recensione` (per `nomeUtente`)
- `ClientHandler` per gestire la sessione utente
- interfacce client per mostrare dati utente

Src → server → service

PreferitiService

PreferitiService gestisce tutte le operazioni relative ai ristoranti preferiti di un utente.

È un livello intermedio tra **ClientHandler** e **PreferitiDAO**, e si occupa di:

- leggere i parametri dal **Request**
- invocare il DAO
- costruire la risposta JSON da inviare al client

Funzione principale

visualizzaPreferiti(Request req)

Carica tutti i ristoranti preferiti di un utente.

Flusso operativo:

1. Legge **idUtente** dal payload.
2. Chiama **PreferitiDAO.getRistorantiPreferiti(idUtente)** per ottenere la lista completa.
3. Per ogni ristorante:
 - costruisce un JSON con tutti i campi principali
 - aggiunge la proprietà "**fonte**" in base a **r.isTheKnife()**
4. Inserisce tutto in un **JSONArray**

5. Restituisce `Response.ok(payload)`

Ruolo nel sistema

È il punto di accesso per la visualizzazione dei preferiti nel client.

Si occupa anche di uniformare i dati provenienti da THEKNIFE e UTENTE.

RecensioneService

`RecensioneService` gestisce l'intero ciclo di vita delle recensioni:

- aggiunta
- modifica
- eliminazione
- recupero per ristorante
- recupero per utente
- recupero per gestore
- gestione recensioni anonime
- gestione recensioni non anonime
- gestione risposte del gestore

È uno dei service più complessi del sistema.

aggiungi(Request req)

Aggiunge una nuova recensione.

Logica chiave:

- legge idUtente, idRistorante, voto, testo
- determina automaticamente la fonte:
 - se il ristorante è UTENTE → "UTENTE"
 - altrimenti → "THEKNIFE"
- verifica se l'utente ha già recensito quel ristorante
- chiama `RecensioneDAO.aggiungi(...)`

- restituisce `ok` o errore

modifica(Request req)

Modifica una recensione esistente.

- legge `idRecensione`, voto, testo
- chiama `RecensioneDAO.modifica(...)`
- restituisce esito

elimina(Request req)

Elimina una recensione.

- legge `idRecensione`
- chiama `RecensioneDAO.elimina(idRecensione)`

getByRistorante(Request req)

Restituisce tutte le recensioni (anonime + non anonime) di un ristorante.

- chiama `RecensioneDAO.getByRistorante(idRistorante)`
- costruisce un JSON con tutti i campi principali

getNonAnonime(Request req)

Restituisce solo le recensioni con nome utente visibile.

Caratteristiche importanti:

- include `nomeUtente`
- include la risposta del gestore (se esiste)
- usa `RecensioneDAO.getRispostaByRecensione(id)`

getByUtente(Request req)

Restituisce tutte le recensioni scritte da un utente.

- include nome del ristorante
- utile per la sezione “Le mie recensioni”

getByGestore(Request req)

Restituisce tutte le recensioni dei ristoranti gestiti da un gestore.

- usa `RecensioneDAO.getByGestore(idGestore)`
- include nome ristorante

rispondi(Request req)

Permette al gestore di rispondere a una recensione.

- legge idRecensione e testo risposta
- chiama `RecensioneDAO.rispondi(...)`

visualizzaRecensioniAnonime(Request req)

Restituisce solo recensioni anonime, con eventuale risposta del gestore.

- include `risposta` se presente
- include `id` per permettere al gestore di rispondere

RistoranteService

RistoranteService è il service più articolato del sistema.

Gestisce:

- ricerca ristoranti (THEKNIFE + UTENTE)
- visualizzazione dettagli
- aggiunta, modifica, eliminazione ristoranti utente
- controllo proprietà
- riepilogo gestore
- recensioni anonime

cerca(Request req)

Esegue la ricerca completa dei ristoranti.

Funzionamento:

1. Legge tutti i filtri dal payload (query, tipo cucina, località, prezzo, delivery, prenotazione, stelle).
2. Cerca nei ristoranti THEKNIFE tramite **RistoranteDAO.cerca(...)**.
3. Cerca nei ristoranti UTENTE tramite **RistoranteUtenteDAO.cerca(...)**.
4. Unisce i risultati in un unico JSON.
5. Imposta "**fonte**" per distinguere le due origini.

È la funzione usata dalla barra di ricerca del client.

controllaProprieta(Request req)

Verifica se un gestore è proprietario di un ristorante.

- usa `RistoranteUtenteDAO.isOwnedBy(idRistorante, idGestore)`
- restituisce ok o errore

visualizza(Request req)

Restituisce i dettagli di un ristorante THEKNIFE.

- usa `RistoranteDAO.getId(id)`
- costruisce JSON completo

visualizzaUtente(Request req)

Restituisce i dettagli di un ristorante UTENTE.

- usa `RistoranteUtenteDAO.getId(id)`

aggiungi(Request req)

Aggiunge un nuovo ristorante creato da un gestore.

- legge tutti i campi dal payload
- chiama `RistoranteUtenteDAO.aggiungi(...)`

modifica(Request req)

Modifica un ristorante utente.

- gestisce campi opzionali e null
- chiama `RistoranteUtenteDAO.modifica(...)`

elimina(Request req)

Elimina un ristorante utente.

- chiama `RistoranteUtenteDAO.elimina(id)`

riepilogoGestore(Request req)

Restituisce tutti i ristoranti gestiti da un gestore.

- usa `RistoranteUtenteDAO.getByGestore(idGestore)`
- costruisce JSON semplificato

visualizzaRecensioniAnonime(Request req)

Restituisce recensioni anonime di un ristorante, con eventuale risposta.

UtenteService

UtenteService gestisce autenticazione e registrazione.

login(Request req)

Effettua il login.

- legge email e password
- chiama `UtenteDAO.login(...)`
- se valido, restituisce:
 - id
 - username
 - email
 - ruolo

Il campo "`username`" è adattato per il client.

registrati(Request req)

Registra un nuovo utente.

Flusso:

1. legge username, email, password, ruolo
2. chiama `UtenteDAO.registrati(...)`
3. recupera l'utente appena registrato
4. restituisce JSON con i dati principali

Src → server → utils

DatabasePopulator

DatabasePopulator è un'utility standalone progettata per importare un file CSV contenente ristoranti Michelin/MyMaps all'interno della tabella **restauranttheknife**.

È pensata per essere eseguita una tantum o per aggiornamenti massivi del database.

Funzione principale

importCSV()

Esegue l'intero processo di importazione.

Flusso operativo:

Definizione della query SQL

1. La query è un **INSERT INTO restauranttheknife (...) VALUES (...)** con 10 campi:
 - nome
 - nazione
 - città
 - indirizzo
 - latitudine
 - longitudine
 - fascia_prezzo

- delivery
- prenotazione
- Tipo_cucina

2. Apertura delle risorse

- Connessione MySQL tramite `DriverManager.getConnection(...)`
- `PreparedStatement` per l'INSERT
- `BufferedReader` per leggere il CSV

3. Disattivazione dell'autocommit

- `conn.setAutoCommit(false)`
- permette di eseguire un batch unico e più efficiente

4. Lettura del CSV

- salta la prima riga (header)
- per ogni riga:
 - parsing robusto tramite `parseCSV()`
 - estrazione dei campi
 - parsing di città e nazione da un unico campo
 - calcolo della fascia prezzo come lunghezza del simbolo €€€
 - conversione di latitudine e longitudine

5. Batch insert

- ogni ristorante viene aggiunto al batch con `ps.addBatch()`
- alla fine: `ps.executeBatch()`

6. Commit finale

- `conn.commit()`
- stampa del numero di record importati

Metodo di supporto

`parseCSV(String line)`

Parser CSV personalizzato che gestisce correttamente:

- campi con virgolette
- campi contenenti virgole interne

- parsing manuale senza dipendenze esterne

Restituisce un array di stringhe con i campi estratti.

Ruolo nel sistema

DatabasePopulator è utilizzato per:

- popolare inizialmente il database THEKNIFE
- aggiornare i dati Michelin/MyMaps
- testare il sistema con dataset reali

Non viene mai invocato dal server in runtime: è uno strumento offline.

DBConnectionPool

`DBConnectionPool` è un pool di connessioni MySQL implementato tramite `ArrayBlockingQueue`.

Serve a migliorare le prestazioni del server evitando di creare una nuova connessione per ogni richiesta.

Struttura

pool

- `ArrayBlockingQueue<Connection>` contenente fino a 10 connessioni MySQL pre-create.

Funzioni principali

`initialize(String host, String dbName, String user, String password)`

Inizializza il pool.

Flusso operativo:

Carica il driver MySQL:

```
java
```

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

1.

Crea la coda con capacità 10:

```
java  
pool = new ArrayBlockingQueue<>(10);
```

2.

Costruisce l'URL JDBC:

Codice

```
jdbc:mysql://host/dbName?useSSL=false&serverTimezone=UTC
```

3.

Crea 10 connessioni:

```
java  
for (i = 0; i < 10; i++)  
    pool.add(DriverManager.getConnection(...));
```

4.

Stampa conferma:

Codice

Pool di connessioni MySQL inizializzato correttamente.

5.

get()

Restituisce una connessione disponibile.

- utilizza `pool.take()`
- se il pool è vuoto, attende finché una connessione non viene rilasciata

release(Connection conn)

Rimette una connessione nel pool.

- usa `pool.offer(conn)`
- ignora connessioni nulle

Ruolo nel sistema

`DBConnectionPool` è fondamentale per:

- ridurre la latenza delle query
- evitare overhead di creazione connessioni
- gestire carichi elevati con più client simultanei

È utilizzato da tutti i DAO del server.

Src → server

ServerMain

`ServerMain` è il punto di ingresso dell'intero server TheKnife.

È responsabile di:

- inizializzare la connessione al database tramite il connection pool
- avviare il listener TCP che accetta le connessioni dei client
- configurare i parametri principali del server (porta, DB, credenziali)

È la classe che viene eseguita quando si avvia il server.

Struttura e responsabilità

Inizializzazione del database

La prima operazione eseguita dal server è l'inizializzazione del pool di connessioni:

```
java
```

```
DBConnectionPool.initialize(host, dbName, user, password);
```

Questo:

- carica il driver MySQL

- crea 10 connessioni persistenti
- le mette nel pool
- rende immediatamente disponibile il database a tutti i DAO

Il blocco commentato mostra che in origine i parametri potevano essere inseriti manualmente tramite `Scanner`, ma ora sono hardcoded per comodità:

```
java
String host = "localhost";
String dbName = "theknife";
String user = "root";
String password = "";
```

Avvio del server

Dopo l'inizializzazione del DB, il server avvia il listener TCP:

```
java
int port = 5555;
ServerListener listener = new ServerListener(port);
listener.start();
```

ServerListener

`ServerListener` (che si trova nel package `server.connection`) è un thread che:

- apre una `ServerSocket` sulla porta indicata
- accetta connessioni dai client
- per ogni client crea un nuovo `ClientHandler`
- gestisce la comunicazione asincrona client → server → client

`ServerMain` quindi non gestisce direttamente le richieste:

si limita ad avviare l'infrastruttura.

Flusso di avvio completo

1. Configurazione parametri DB
2. Inizializzazione del connection pool
3. Stampa messaggio di avvio
4. Creazione del listener TCP
5. Start del listener
6. Il server è operativo e pronto a ricevere richieste

Ruolo nel sistema

ServerMain è il cuore dell'applicazione lato server.

Senza di esso:

- il database non sarebbe accessibile
- nessun client potrebbe collegarsi
- nessun service o DAO potrebbe funzionare

È la classe che si esegue per avviare l'intero backend.